[Dat Nguyen] - Socket Programming - TCP/UDP Sockets & I/O Multiplexing Socket programming

Introduction Socket

Sockets are a method of IPC that allow data to be exchanged between applications, either on the same host (computer) or on different hosts connected by a network

Communication domains

Sockets exist in a communication domain, which determines:

- The method of identifying a socket (i.e., the format of a socket “address”)
- The range of communication (i.e., either between applications on the same host or between applications on different hosts connected via a network)

Domain	Communication performed	Communication between applications	Address format	Address structure
`AF_UNIX`	within kernel	on same host	pathname	`sockaddr_un`
`AF_INET`	via IPv4	on hosts connected via an IPv4 network	32-bit IPv4 address + 16-bit port number	`sockaddr_in`
`AF_INET6`	via IPv6	on hosts connected via an IPv6 network	128-bit IPv6 address + 16-bit port number	`sockaddr_in6`

Socket system calls

System Call	Description
`socket()`	Creates a new socket.

System Call	Description
`bind()`	Binds a socket to a specific address. A server usually uses this to bind its socket to a well-known address so that clients can locate it.
`listen()`	Marks a stream socket as passive, allowing it to accept incoming connection requests.
`accept()`	Accepts an incoming connection from a peer on a listening stream socket. It can optionally return the address of the peer socket.
`connect()`	Establishes a connection with another socket, typically used by a client to connect to a server.

---

1. Protocol Independence, OSI Model

1.1 Protocol Inderpendence

Write source code so that the application can work normally with any network protocol (especially IPv4 or IPv6) without having to change or rewrite the code logic

Protocol Dependent

developers often tied the code directly to a specific protocol (for example, IPv4)

```
// Hardcoded to use IPv4
int fd = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
inet_pton(AF_INET, "192.168.1.1", &addr.sin_addr);
```

=> Disadvantage: When you want to upgrade the software to run IPv6 (or support both), you must rewrite all variable creations, data structures, and setup functions.

Protocol Inderpendence

With the protocol-independent approach, the program does not care if the given address is IPv4, IPv6, or another network protocol.

```
struct addrinfo hints, *res;
memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC; // AF_UNSPEC means accept BOTH IPv4 and IPv6!
```

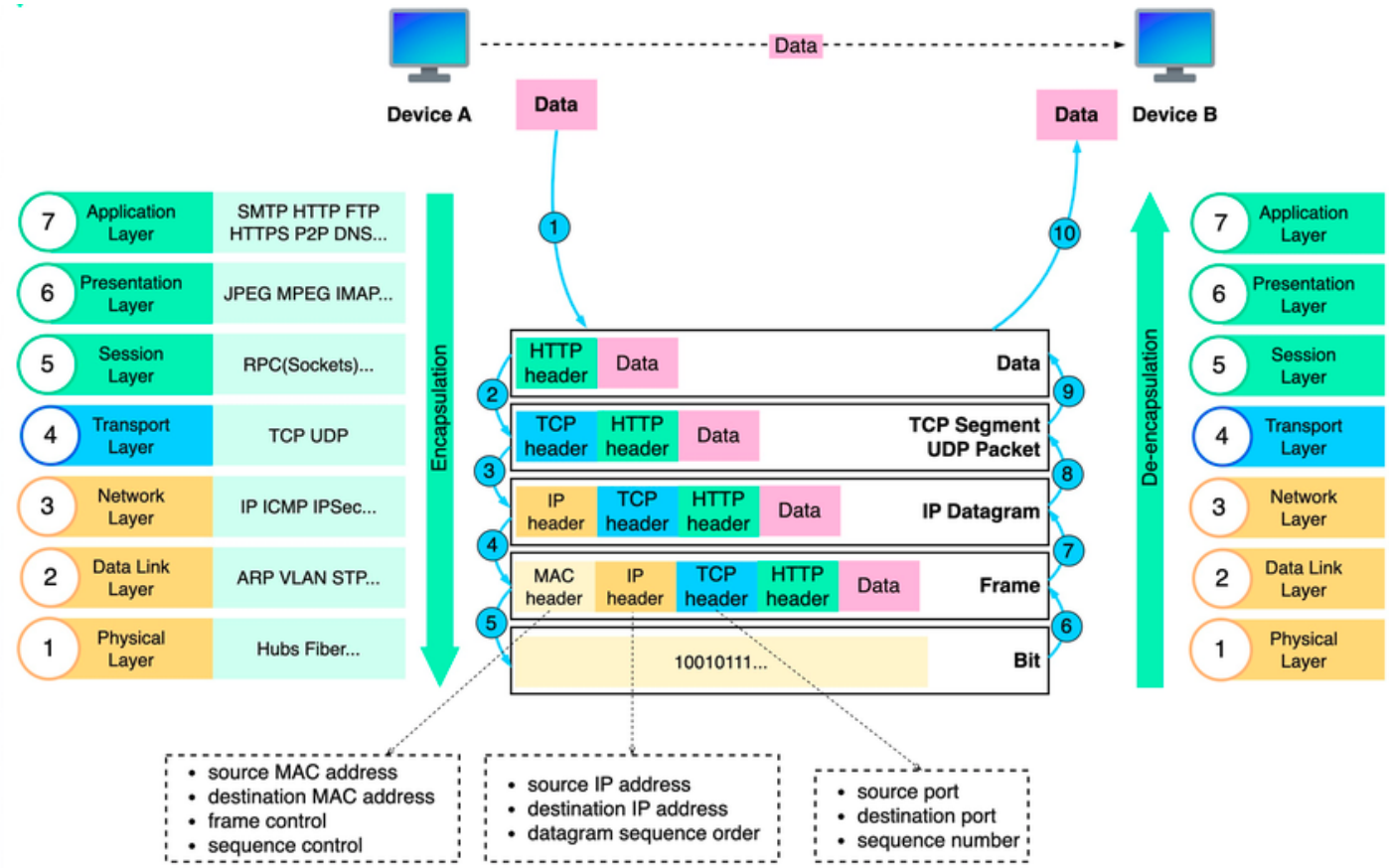
```
hints.ai_socktype = SOCK_STREAM;

// getaddrinfo automatically decides whether the IP/Domain is IPv4 or IPv6
getaddrinfo("localhost", "8080", &hints, &res);

// Automatically create socket and connect based on the returned result, without caring if it is v4 or v6
int fd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
connect(fd, res->ai_addr, res->ai_addrlen);
```

=> Advantages: The same code can run smoothly on both IPv4 and IPv6 networks without changing a single line of code.

1.2 OSI Model



alt text

Step	Process	Description
1	Application Layer	Device A creates the application data and adds an **HTTP header**.

Step	Process	Description
2	Transport Layer	A **TCP or UDP header** is added. The data is encapsulated into a **TCP segment** or **UDP datagram**. The header contains information such as source port and destination port. TCP also contains a sequence number.
3	Network Layer	An **IP header** is added. It contains the **source and destination IP addresses**. The result is an **IP datagram**.
4	Data Link Layer	A **MAC header** (and usually a trailer/FCS) is added. It contains the **source and destination MAC addresses**. The result is a **frame**.
5	Physical Layer	The frame is converted into **bits/signals** and transmitted over the physical medium.
6–10	Device B — De-encapsulation	Device B receives the bits and performs the reverse process. Each layer processes and removes its corresponding header/trailer until the original application data is delivered to the application.

2. Socket Address Structures, Value-Result Arguments, Byte Ordering Functions

2.1 Socket Address Structures

Most socket functions require a pointer to a socket address structure as an argument. Each supported protocol suite defines its own socket address structure. The names of these structures begin with sockaddr_ and end with a unique suffix for each protocol suite.

IPv4 Socket Address Structure

```
struct in_addr {
    in_addr_t s_addr;   /* 32-bit IPv4 address */
                        /* network byte ordered */
};

struct sockaddr_in {
    uint8_t         sin_len;       /* length of structure (16) */
```

```
    sa_family_t     sin_family;    /* AF_INET */
    in_port_t       sin_port;      /* 16-bit TCP or UDP port number */
                                   /* network byte ordered */
    struct in_addr  sin_addr;      /* 32-bit IPv4 address */
                                   /* network byte ordered */
    char            sin_zero[8];   /* unused */
}
```

Generic Socket Address Structure

```
struct sockaddr {
    uint8_t      sa_len;
    sa_family_t  sa_family;    /* address family: AF_xxx value */
    char         sa_data[14];  /* protocol-specific address */thức */
};
```

IPv6 Socket Address Structure

```
struct in6_addr {
    uint8_t s6_addr[16];      /* 128-bit IPv6 address */
                              /* network byte ordered */
};

#define SIN6_LEN              /* required for compile-time tests */

struct sockaddr_in6 {
    uint8_t         sin6_len;      /* length of this struct (28) */
    sa_family_t     sin6_family;   /* AF_INET6 */
    in_port_t       sin6_port;     /* transport layer port# */
                                   /* network byte ordered */
    uint32_t        sin6_flowinfo; /* flow information, undefined */
    struct in6_addr sin6_addr;     /* IPv6 address */
                                   /* network byte ordered */
    uint32_t        sin6_scope_id; /* set of interfaces for a scope */
};
```

New Generic Socket Address Structure

```
struct sockaddr_storage {
    uint8_t      ss_len;     /* length of this struct (implementation dependent) */
    sa_family_t  ss_family;  /* address family: AF_xxx value */
```

```
    /* implementation-dependent elements to provide:
     * a) alignment sufficient to fulfill the alignment requirements of
     *    all socket address types that the system supports.
     * b) enough storage to hold any type of socket address that the
     *    system supports.
     */
};
```

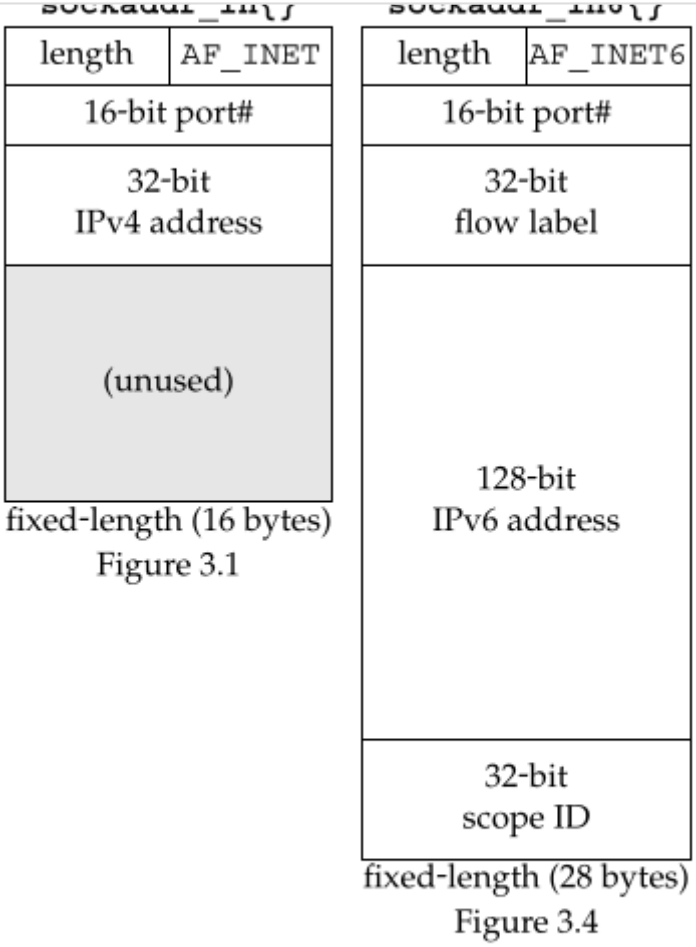


Figure 3.1

Figure 3.4

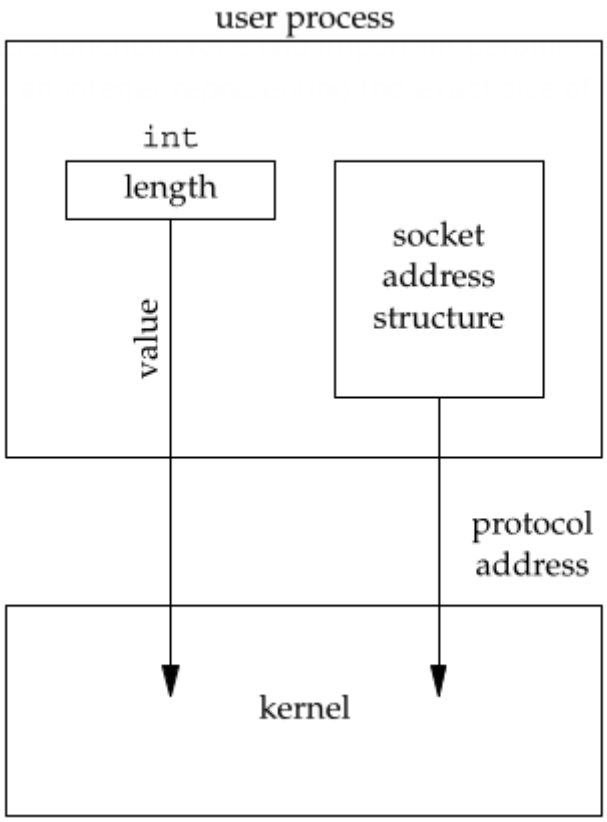
alt text

2.2 Value-Result Argument

General Principles of Passing Socket Address Structures

- Socket address structures are always passed by reference, meaning a pointer to the structure is passed.
- The size (length) of the structure must also be passed as an argument.
- The size is passed depends on the direction of data transfer: Process to Kernel, or Kernel to Process.

Process to Kernel (connect, bind, sendto)

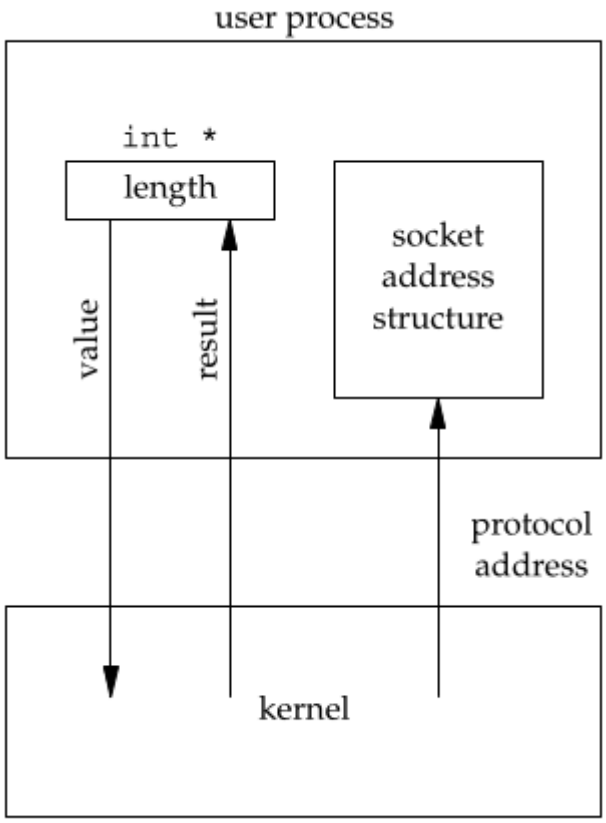


pointer pointing to the socket address structure
structure.

alt text

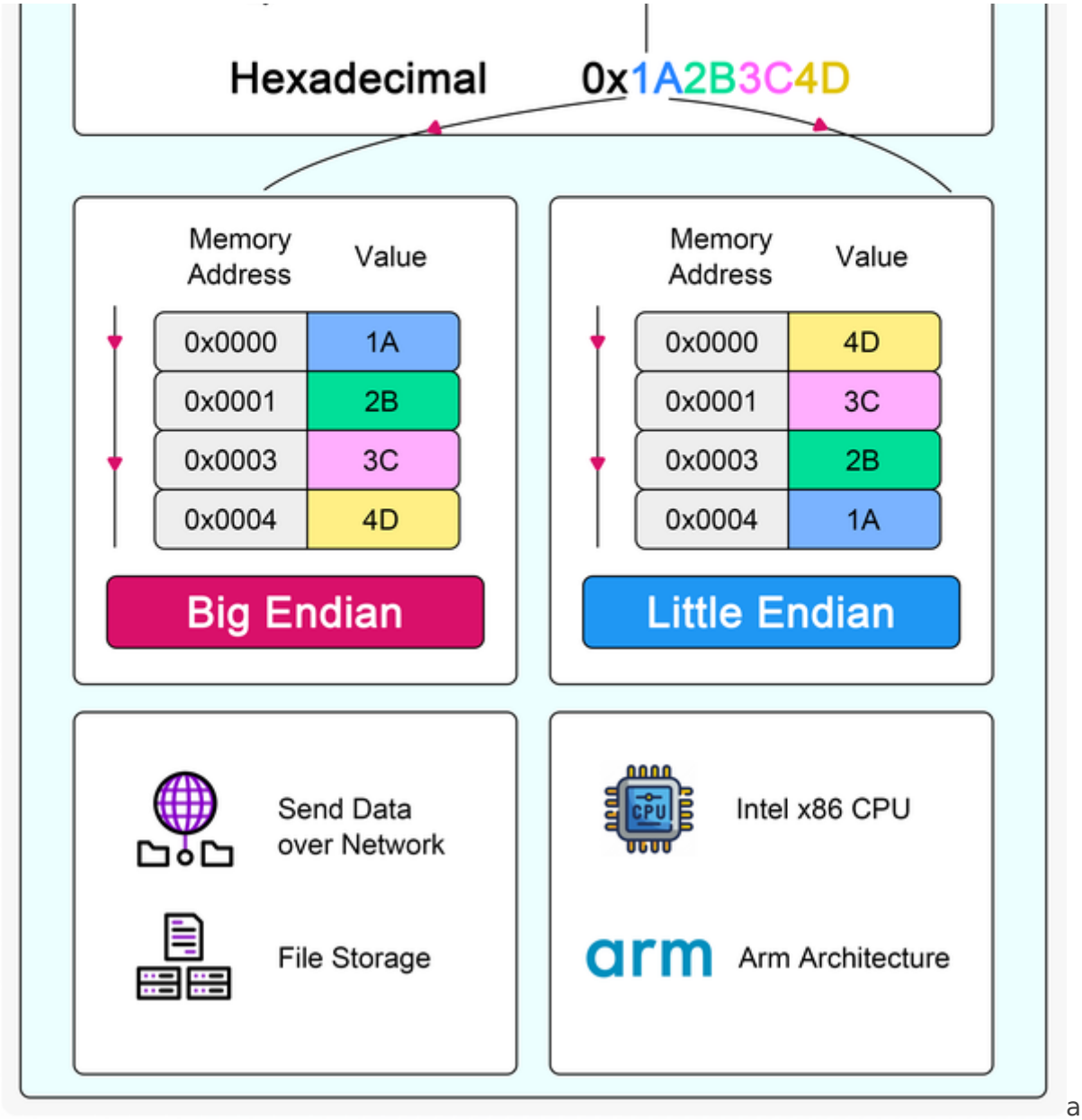
```
struct sockaddr_in
serv;
/* fill in serv{} */
connect(sockfd, (SA *) &serv, sizeof(serv));
```

Kernel to Process (accept, getsockname, recvfrom)




```
struct sockaddr_un cli;
socklen_t len;           /* Unix domain */
len = sizeof(cli);       /* len is a value */
getpeername(unixfd, (SA *) &cli, &len);
/* len may have changed */
```

2.3 Byte Ordering



alt text

Internet Protocols define Big-Endian as the standard byte order on the network (Network Byte Order).

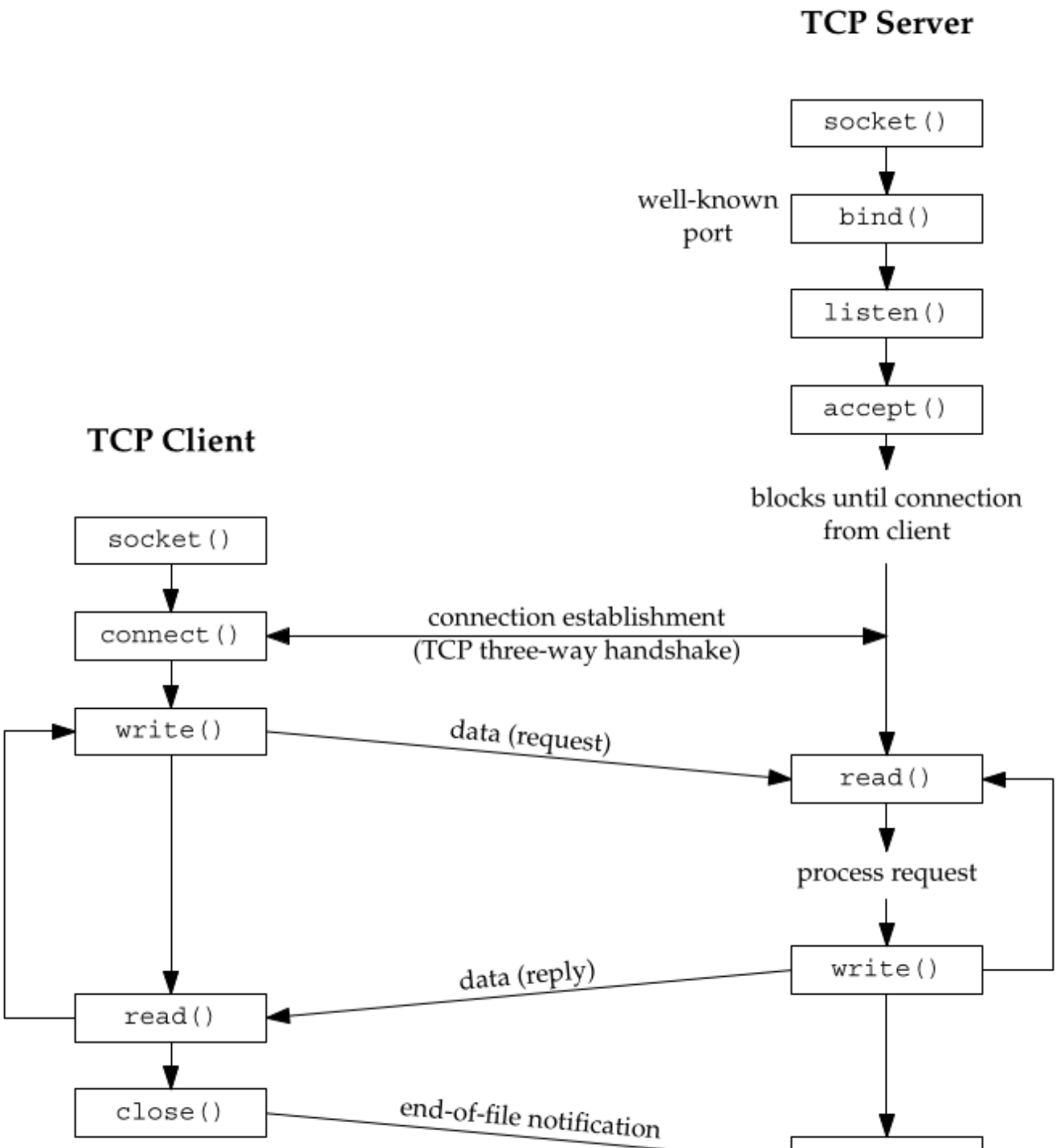
Function	Meaning
`htons()`	Host → Network, 16-bit
`htonl()`	Host → Network, 32-bit
`ntohs()`	Network → Host, 16-bit

Function	Meaning
`ntohl()`	Network → Host, 32-bit

Note

- On Big-Endian systems, these functions are usually defined as empty macros (no processing overhead).
- On Little-Endian systems, the function automatically reverses the corresponding byte order.

3. socket / connect / bind / listen / accept Functions



alt text

3.1 socket Functions

To perform network I/O, the first thing a process must do is call the socket function, specifying the type of communication protocol desired (TCP using IPv4, UDP using IPv6, Unix domain stream protocol, etc.).

```
int socket(int family, int type, int protocol);
```

Family	Description
`AF_INET`	IPv4 protocols
`AF_INET6`	IPv6 protocols
`AF_LOCAL`	Unix domain protocols

Type	Description
`SOCK_STREAM`	Stream socket
`SOCK_DGRAM`	Datagram socket

Protocol	Description
`IPPROTO_TCP`	TCP transport protocol
`IPPROTO_UDP`	UDP transport protocol

3.2 connect Function

The connect function is used by a TCP client to establish a connection with a TCP server

```
int connect(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
```

TCP three-way handshake

```
Client                              Server
  |                                    |
  |──────── SYN ──────────────────────>|  1
  |                                    |
  |<──── SYN + ACK ──────────────────  |  2
  |                                    |
  |──────── ACK ──────────────────────>|  3
  |                                    |
```

```
|         Connection established      |
```

1. Connection Timeout Error (ETIMEDOUT)
   Cause: The client sends a SYN packet but receives no response from the server.
   Mechanism: The kernel retries sending the SYN packet at increasing time intervals (e.g., retrying after 6 seconds, 24 seconds, etc.). If no response is received after a fixed period
2. Connection Refused Error (ECONNREFUSED)
   Cause: The server immediately responds with a RST (Reset) packet. This occurs when the SYN packet arrives at the correct host, but no server process is listening on the requested port number.
   Characteristics: This is a hard error; connect() fails immediately without waiting for a timeout.
3. Host/Network Unreachable Error (EHOSTUNREACH / ENETUNREACH)
   Cause: The SYN packet encounters an issue along the network path, and an intermediate router returns an ICMP "destination unreachable" message.
   Mechanism: The client kernel saves this ICMP message as a soft error and continues retrying the SYN packet. If the timeout expires without success, the kernel officially returns EHOSTUNREACH or ENETUNREACH to the application.

3.3 bind Function

The bind function assigns a local protocol address to a socket. With the Internet proto-cols, the protocol address is the combination of either a 32-bit IPv4 address or a 128-bit IPv6 address, along with a 16-bit TCP or UDP port number.

```
int bind(int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);
```

Parameter	Type	Description
`sockfd`	`int`	Socket file descriptor returned by `socket()`.
`myaddr`	`const struct sockaddr *`	Pointer to the target address structure containing the IP address and port.
`addrlen`	`socklen_t`	Size in bytes of the address structure, usually `sizeof(...)`.

IP Address and Port Configuration

When setting the IP (`sin_addr` / `sin6_addr`) and Port (`sin_port` / `sin6_port`) fields before calling `bind()`:

Specified IP Address	Specified Port	Kernel Behavior
Wildcard	`0`	Kernel chooses **both the IP address and Port**.

Specified IP Address	Specified Port	Kernel Behavior
Wildcard	Non-zero	Kernel chooses the **IP address** and uses the **Port specified by the process**.
Specific IP	`0`	Uses the **IP specified by the process**, while the Kernel chooses the **Port**.
Specific IP	Non-zero	Uses **exactly the IP address and Port specified by the process**.

3.4 listen Functions

The listen function is called only by a TCP server and it performs two actions:

- Socket State Conversion: Change the socket from active to passive to tell the kernel to accept incoming connection requests to this socket (change from CLOSED to LISTEN state).
- Backlog Parameter Setting: Define the maximum number of incoming connections that the kernel can keep in the queue for this socket.

```
int listen(int sockfd, int backlog);
```

3.5 accept Funtions

The TCP server calls accept to get the next finished connection from the front of the completed connection queue.

Blocking Mechanism: By default, if the completed connection queue is empty, the process will sleep until a new connection arrives.

```
int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen)
```

Item	Description
Return Value	Returns a new socket descriptor for the connection with the client, or `-1` if an error occurs.
`cliaddr` **(Structure Pointer)**	Pointer to a structure used to store the protocol address (IP address and Port) of the connected client.
`addrlen` **(Value-Result Parameter)**	**Before the call:** Set to the size of the address structure pointed to by `cliaddr`. **After the call:** The kernel stores the actual size (in bytes) of the client address in this variable.

Difference Between Listening Socket and Connected Socket: This is the most important point when handling server connections:

Listening Socket:

- A descriptor created by the socket() function, then used as a parameter for bind() and listen().
- A typical server only creates one Listening Socket, and it stays alive throughout the server application's lifetime.

Connected Socket:

- Automatically created and returned by the kernel every time the accept() function succeeds (after completing the TCP 3-way handshake).
- Represents a separate communication channel between the server and that specific client.

After the server finishes working with the client: The Connected Socket is closed, while the Listening Socket stays open to wait for other connections.

3.6 Example

Server : socket() -> bind() -> listen() -> accept() -> read/write

```
#define PORT 8080
#define BUFFER_SIZE 1024

int main(void)
{
    int server_fd;
    int client_fd;

    struct sockaddr_in server_addr;
    struct sockaddr_in client_addr;

    socklen_t client_len = sizeof(client_addr);

    char buffer[BUFFER_SIZE];

    /* 1. socket() */
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```
if (server_fd == -1)
{
    perror("socket");
    return 1;
}

/* 2. bind() */
memset(&server_addr, 0, sizeof(server_addr));

server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);

if (bind(server_fd,
         (struct sockaddr *)&server_addr,
         sizeof(server_addr)) == -1)
{
    perror("bind");
    close(server_fd);
    return 1;
}

/* 3. listen() */
if (listen(server_fd, 5) == -1)
{
    perror("listen");
    close(server_fd);
    return 1;
}

printf("Server is listening on port %d...\n", PORT);

/* 4. accept() */
client_fd = accept(
    server_fd,
    (struct sockaddr *)&client_addr,
    &client_len
);

if (client_fd == -1)
```

```
    {
        perror("accept");
        close(server_fd);
        return 1;
    }

    char client_ip[INET_ADDRSTRLEN];

    inet_ntop(
        AF_INET,
        &client_addr.sin_addr,
        client_ip,
        sizeof(client_ip)
    );

    printf("Client connected!\n");
    printf("Client IP   : %s\n", client_ip);
    printf("Client PORT : %d\n", ntohs(client_addr.sin_port));
    /* Receive message */
    memset(buffer, 0, sizeof(buffer));

    int n = read(client_fd, buffer, sizeof(buffer) - 1);

    if (n > 0)
    {
        buffer[n] = '\0';

        printf("Client says: %s\n", buffer);

        /* Send response */
        const char *response = "Hello from server!";

        write(client_fd, response, strlen(response));
    }

    close(client_fd);
    close(server_fd);

    return 0;
}
```

Client: socket() -> connect() -> read/write

```
#define PORT 8080
#define BUFFER_SIZE 1024

int main(void)
{
    int sock_fd;

    struct sockaddr_in server_addr;

    char buffer[BUFFER_SIZE];

    /* 1. socket() */
    sock_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (sock_fd == -1)
    {
        perror("socket");
        return 1;
    }

    /* 2. Prepare server address */
    memset(&server_addr, 0, sizeof(server_addr));

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);

    inet_pton(
        AF_INET,
        "127.0.0.1",
        &server_addr.sin_addr
    );

    /* 3. connect() */
    if (connect(
            sock_fd,
            (struct sockaddr *)&server_addr,
```

```
            sizeof(server_addr)) == -1)
    {
        perror("connect");
        close(sock_fd);
        return 1;
    }

    printf("Connected to server!\n");

    /* Send message */
    const char *message = "Hello from client!";

    write(sock_fd, message, strlen(message));

    /* Receive response */
    memset(buffer, 0, sizeof(buffer));

    int n = read(sock_fd, buffer, sizeof(buffer) - 1);

    if (n > 0)
    {
        buffer[n] = '\0';

        printf("Server says: %s\n", buffer);
    }

    close(sock_fd);

    return 0;
}
```

Output

```
terminal 1:
./server
Server is listening on port 8080...

terminal 2:
./client
Connected to server!
```

```
Server says: Hello from server!

terminal 1:
Client connected!
Client IP   : 127.0.0.1
Client PORT : 47956
Client says: Hello from client!
```

4 Concurrent Servers (fork-per-connection)

Iterative Server: Handles only one client at a time. If a request takes a long time, other clients have to wait and get blocked (not suitable for real servers with high loads).

Concurrent Server: Allows handling multiple clients at the same time. The simplest way on Unix systems is to call the fork function to create a child process dedicated to handling each client.

The outline for a typical concurrent server:

```
pid_t   pid;
int     listenfd, connfd;

listenfd = Socket( ... );

/* fill in sockaddr_in{} with server's well-known port */
Bind(listenfd, ... );
Listen(listenfd, LISTENQ);

for ( ; ; ) {
    connfd = Accept(listenfd, ... );    /* probably blocks */

    if ( (pid = Fork()) == 0) {
        Close(listenfd);    /* child closes listening socket */
        doit(connfd);       /* process the request */
        Close(connfd);      /* done with this client */
        exit(0);            /* child terminates */
    }

    Close(connfd);          /* parent closes connected socket */
```

```
}
```

Basic Workflow of a Concurrent Server:

The server creates listenfd, configures bind, and calls listen to wait for connections.

In an infinite loop (for (; ;)), the server calls accept to receive a connection from a client (connfd).

Right after accepting the connection, the server calls fork():

- Child Process: Closes listenfd (because it does not need to listen for new connections), calls the client service function (doit(connfd)), then closes connfd and exits (exit(0)).
- Parent Process: Closes connfd (because the child process takes care of serving this client), then goes back to the loop to continue calling accept and wait for the next client on listenfd.

5 close / getsockname / getpeername Functions

5.1 close Funtions

Used to close the socket and terminate the TCP connection.

```
int close(int sockfd);
```

Behavior:

- Marks the socket as closed and returns immediately. The socket descriptor cannot be used again for read or write commands.
- TCP will try to send all remaining queued data, then perform the normal TCP connection termination process.

5.2 getsockname Funtions

getsockname: Used to return the local protocol address (IP address and port number) associated with a socket.

```
int getsockname(int sockfd, struct sockaddr *localaddr, socklen_t *addrlen);
```

When getsockname is needed:

- When a TCP client does not call bind, after a successful connect, the client can use getsockname to get the local IP address and port number automatically assigned by the

kernel.
- After calling bind with a port number of 0 (asking the kernel to choose a port), this function helps retrieve the exact assigned local port number.
- Used to determine the address family of a socket.
- When a TCP server binds to a wildcard IP address, after a connection is established (accept returns successfully), the server can call getsockname on the connected socket (not the listening socket) to find out the actual local IP address of the connection.

5.3 getpeername Fuctions

getpeername: Used to return the protocol address of the remote side (foreign protocol address, which is the client) associated with the socket.

```
int getpeername(int sockfd, struct sockaddr *peeraddr, socklen_t *addrlen);
```

When getpeername is needed:

- When a server runs through the exec function from a process that called accept (such as how inetd forks and execs a TCP server). Because exec completely replaces the memory space of the child process (causing the client address structure to be lost), but the socket descriptor remains intact across exec, the new server must call getpeername to identify the client's IP address and port number.

6 TCP Echo Server & Client walkthrough (str_echo/str_cli)

6.1 str_echo

Read Data: The server receives and reads data sent by the client through the socket.

Send Back (writen): Right after finishing reading, the server uses the writen function to send that exact line of data back to the client.

```
if ( (childpid = Fork()) == 0) {    /* child process */
    Close(listenfd);                /* close listening socket */
    str_echo(connfd);               /* process the request */
    exit(0);
}
```

Handling When the Client Closes the Connection: When the client actively closes the connection, receiving the FIN packet causes the read function to return 0. This makes the str_echo function end, which then causes the child process to terminate.

6.2 str_cli

Outgoing Direction: It reads the text you type from the keyboard -> then writes it to the socket to send it up to the server.

Incoming Direction: It reads from the socket (gets the data returned by the server) -> then prints it out on the computer screen for you to see the result.

```
void
str_cli(FILE *fp, int sockfd)
{
   char    sendline[MAXLINE], recvline[MAXLINE];

   while (Fgets(sendline, MAXLINE, fp) != NULL) {
       Writen(sockfd, sendline, strlen(sendline));

       if (Readline(sockfd, recvline, MAXLINE) == 0)
           err_quit("str_cli: server terminated prematurely");

       Fputs(recvline, stdout);
   }
}
```

Sending Data (fgets & writen): The fgets function reads a line of text from fp, and then the writen function sends all that data through the socket (sockfd) to the server.

Receiving Response (readline & fputs): The readline function reads the line of data that the server just echoed back through the socket, and the fputs function outputs that content to the screen.

Loop Termination Condition: The while loop ends when fgets returns a null pointer. This happens when the user types an end-of-file signal (EOF, such as pressing Ctrl+D on Linux) or an error occurs.

6.3 Compare str_echo and str_cli

Comparison Criteria	`str_cli` (Client Side)	`str_echo` (Server Side)
Execution Location	Runs in the **Client** program.	Runs in the **Server** program, usually handled by a child process.
Input Source	Reads data from the **user's keyboard** using `fgets()` from `stdin`.	Reads data from the **network socket** using `read()`.

Comparison Criteria	`str_cli` (Client Side)	`str_echo` (Server Side)
Data Processing / Output	Sends the input data through the socket to the server, then receives the response and **prints it to the screen** using `fputs()` to `stdout`.	Receives data from the socket and **immediately sends it back through the same socket** using `writen()`.
Loop Termination	Terminates when the user enters **EOF** (e.g., presses `Ctrl+D`), causing `fgets()` to return `NULL`.	Terminates when the client closes the connection, causing `read()` to return `0` after receiving the TCP `FIN`.

str_cli: The bridge between Human <-> Network (Type -> Send -> Receive -> Print).

str_echo: A pure Network -> Network machine on the server side (Receive from socket -> Echo back to socket).

8. Normal Termination, SIGPIPE Signal, server/client crash scenarios

8.1 Normal Termination

Ctrl+D : End Of File signal

8.2 SIGPIPE Signal

Cause: When a process tries to write data to a socket whose peer has died (and the socket has received a RST packet), the operating system sends a SIGPIPE signal to that process.

Default Behavior: If unhandled, the default behavior of this signal is to terminate the process immediately.

EPIPE Error: If configure the program to ignore SIGPIPE or catch it and return, the write function will not crash the program; instead, it returns an error value with the global error code EPIPE.

8.3 Code Exmamle Server - client (LAB 1)

Server:

```
//Server Echo Function (str_echo)
void str_echo(int sockfd) {
    char buf[1024];
    int n;
```

```
    while (1) {
        n = read(sockfd, buf, sizeof(buf));
        if (n <= 0)
            break; // Exit loop upon encountering EOF or an error

        write(sockfd, buf, n);
    }
}

//Signal Handler for Child Processes (sig_chld)
void sig_chld(int signo){
    pid_t pid;
    int stat;
    while ((pid = waitpid(-1, &stat, WNOHANG)) > 0); // Clean up zombie processes
asynchronously
}

int main()
{
    int server_fd, client_fd;
    struct sockaddr_in server_addr;
    pid_t pid;

    server_fd = socket(AF_INET, SOCK_STREAM, 0); // Create a TCP socket

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(8080);          // Set port to 8080
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1"); // Bind to localhost

    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)); // Bind socket to
address
    listen(server_fd, 5); // Start listening for incoming connections with a backlog of 5

    signal(SIGCHLD, sig_chld); // Register signal handler to prevent zombie processes

    printf("Server listening on 127.0.0.1:8080\n");

    while(1)
    {
        client_fd = accept(server_fd, NULL, NULL); // Accept an incoming client connection
```

```
        if(client_fd < 0){
            continue;
        }

        pid = fork(); // Create a new child process for the accepted client
        if(pid < 0){
            perror("Fork failed");
            close(client_fd);
            continue;
        }
        else if(pid == 0){ // Child process block
            close(server_fd); // Child closes the listening socket descriptor

            // Use getpeername to look up client information
            struct sockaddr_in peer_addr;
            socklen_t peer_len = sizeof(peer_addr);
            if(getpeername(client_fd, (struct sockaddr *)&peer_addr, &peer_len) == 0){
                printf("[Server Child] servicing Client from IP: %s, Port: %d\n",
                       inet_ntoa(peer_addr.sin_addr), ntohs(peer_addr.sin_port));
            }

            str_echo(client_fd); // Run the echo service for the client
            close(client_fd);    // Close the connected socket
            exit(0);             // Terminate the child process
        }
        close(client_fd); // Parent process closes the client descriptor (handled by child)
    }

    close(server_fd);
    return 0;
}
```

client

```
// Client communication function (reads from keyboard, sends to server, receives echo, and
prints to screen)
void str_cli(FILE *fp, int sockfd) {
    char sendline[1024], recvline[1024];

    while (1) {
```

```
        printf("Input: ");
        if (fgets(sendline, sizeof(sendline), fp) == NULL)
            break; // Stop if EOF (Ctrl+D) is received from the keyboard

        // Send data to the server
        ssize_t w_bytes = write(sockfd, sendline, strlen(sendline));
        if (w_bytes < 0) {
            if (errno == EPIPE) {
                fprintf(stderr, "\n[Client Error] EPIPE Error: Server has shut down,
connection broken (Broken pipe)!\n");
            } else {
                perror("\n[Client Error] Other write error");
            }
            break;
        }

        // Receive the echoed data returned from the server
        int n = read(sockfd, recvline, sizeof(recvline) - 1);
        if (n <= 0) {
            fprintf(stderr, "str_cli: server terminated prematurely\n");
            break;
        }

        recvline[n] = '\0';
        printf("Echo: %s", recvline);
    }
}

int main()
{
    int sockfd;
    struct sockaddr_in server_addr;

    signal(SIGPIPE, SIG_IGN); // Ignore SIGPIPE to prevent crash when writing to a closed
connection

    sockfd = socket(AF_INET, SOCK_STREAM, 0); // Create a TCP socket

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(8080);          // Set port to 8080
```

```
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr); // Set server IP address to
localhost

    connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)); // Establish TCP
connection with server

    printf("Connected to server\n");

    // Use getsockname to inspect the client's own local IP and Port assigned by the kernel
    struct sockaddr_in local_addr;
    socklen_t addr_len = sizeof(local_addr);
    if(getsockname(sockfd, (struct sockaddr *)&local_addr, &addr_len) == 0){
        printf("[Info] Client used Local IP: %s, Port: %d\n",
               inet_ntoa(local_addr.sin_addr), ntohs(local_addr.sin_port));
    }

    // Call str_cli passing standard input and the socket file descriptor
    str_cli(stdin, sockfd);

    close(sockfd); // Close the socket descriptor

    return 0;
}
```

Output

Terminal 1

```
./server

Server listening on 127.0.0.1:8080
[Server Child] servicing Client from IP: 127.0.0.1, Port: 37012
```

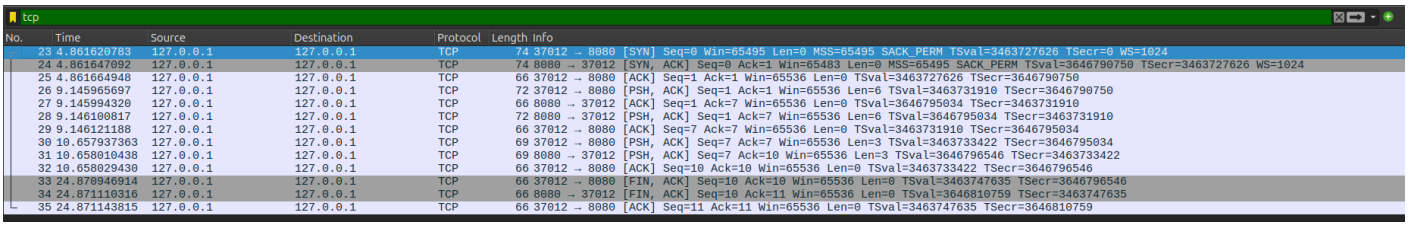
Terminal 2

```
./client

Connected to server
[Info] Client used Local IP: 127.0.0.1, Port: 37012
Input: hello
Echo: hello
```

```
Input: hi
Echo: hi
Input: ^C
```

Terminal 2 : Wireshark



8.4 server/client crash scenarios

8.4.1 Crashing of Server Host

Initial Silence: When the server crashes suddenly (rather than being shut down gracefully by an operator), it cannot send any notification or packets into the network to inform the client.

Client Sends Data: If the client sends a new line of data using the writen function, TCP still packages and sends this data normally. Afterward, the client gets blocked at the readline function waiting for a response (echo) from the server.

Retransmission Process: Because the server has crashed and does not send back an acknowledgment packet (ACK), the TCP on the client side will continuously retransmit the data packet. According to Berkeley-derived systems, the client will retry 12 times over a period of about 9 minutes before giving up.

- Setting a Timeout for readline: Helps the client avoid hanging for a full 9 minutes, allowing it to disconnect sooner when a failure occurs.

8.4.2 Crashing and Rebooting of Server Host

Startup and Connection: The client and server successfully establish the initial connection.

Server Crash and Reboot: The server crashes, reboots, and reconnects to the network while the client is idle (not sending any data). Because there is no keep-alive mechanism, the client has no idea that the server has crashed and restarted.

Client Sends Data: The user types a command line on the client side, and the data is packaged by TCP and sent to the server.

Server Responds with an RST Packet: Because the server just rebooted, all states and information about old connections have been completely wiped from its kernel memory. Upon receiving a data packet from a connection it no longer tracks, the server's TCP immediately responds with an RST (Reset) packet.

readline Returns the ECONNRESET Error: The client is currently blocked at the readline function. When it receives the RST packet from the server, readline is immediately interrupted and returns the ECONNRESET error code.

Note:

Undetected Server Reboot: Similar to the complete server crash scenario, if the client does not actively send data, it cannot automatically detect that the server has rebooted.

Solutions for High Real-Time Requirements: To solve this issue, developers must use supporting mechanisms such as:

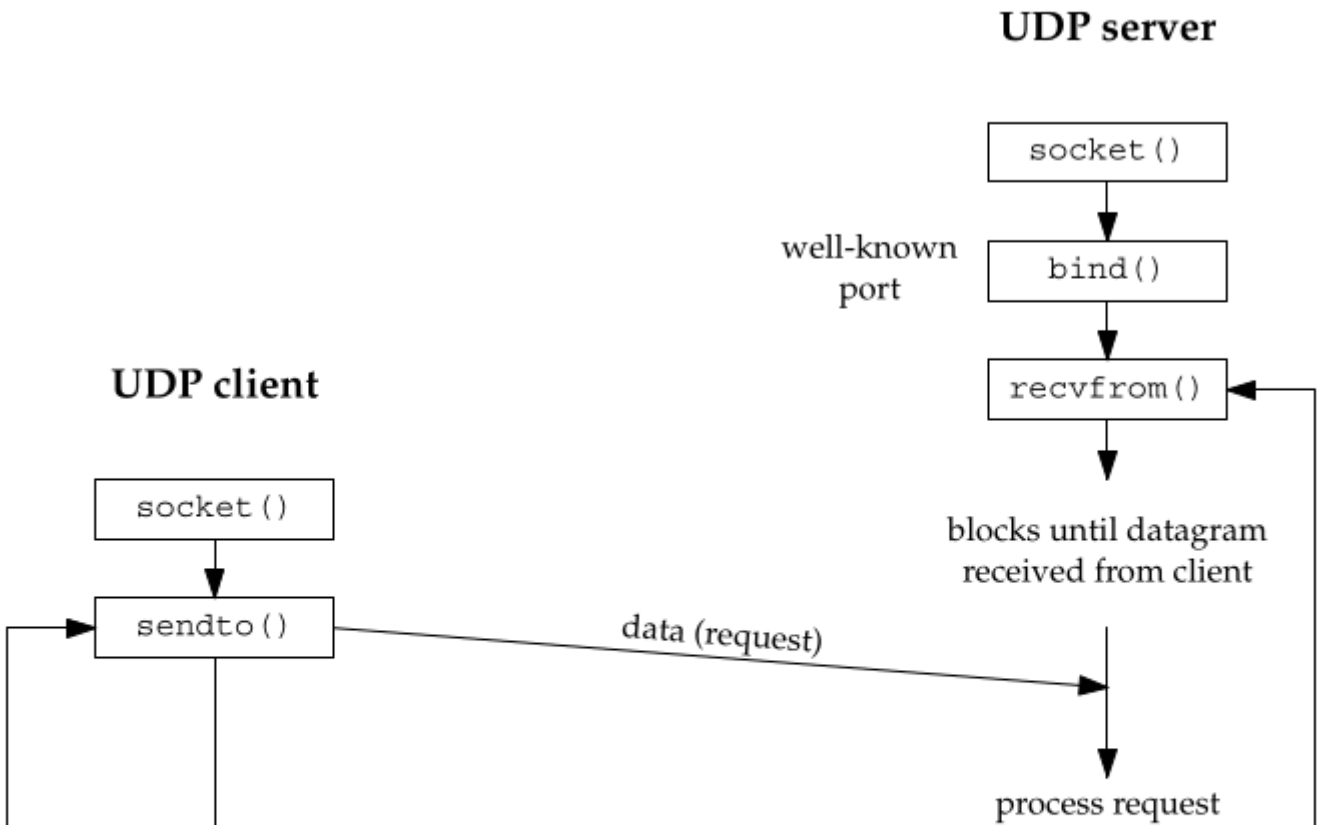
- The SO_KEEPALIVE socket option.
- Building heartbeat mechanisms (periodic connection-checking pulses) between the client and server.

9. UDP: recvfrom/sendto, UDP Echo Server/Client, Lost Datagrams, connect() with UDP, lack of Flow Control

Basic Difference Between TCP and UDP:

```
UDP: A datagram-based protocol, connectionless, and unreliable.


TCP: A reliable byte-stream, connection-oriented protocol.
```



alt text

9.1 recvfrom/sendto Funtions

```
ssize_t recvfrom(int sockfd, void *buff, size_t nbytes, int flags,
                 struct sockaddr *from, socklen_t *addrlen);

to: A pointer to the struct sockaddr structure containing the destination address (IP + Port)
that you want to send the packet to
```

```
ssize_t sendto(int sockfd, const void *buff, size_t nbytes, int flags,
               const struct sockaddr *to, socklen_t addrlen);

from: A pointer to the struct sockaddr structure that the function will automatically fill in
with the address of the sender who sent the packet.
```

First 3 Parameters (sockfd, buff, nbytes): Identical to the standard read and write function parameters:

- sockfd: The socket descriptor.
- buff: A pointer to the buffer memory containing the data to read or write.
- nbytes: The size (number of bytes) to read or write.

9.2 UDP Echo Server/Client

9.2.1 UDP Echo Server

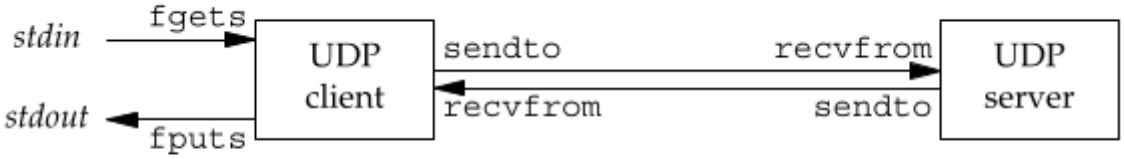


Figure 8.2 Simple echo client/server using UDP.

alt text

```
int main(int argc, char **argv)
{
    int sockfd;
    struct sockaddr_in servaddr, cliaddr;

    sockfd = Socket(AF_INET, SOCK_DGRAM, 0);

    bzero(&servaddr, sizeof(servaddr));
```

```
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);

    Bind(sockfd, (SA *) &servaddr, sizeof(servaddr));

    dg_echo(sockfd, (SA *) &cliaddr, sizeof(cliaddr));
}
```

The function dg_echo is called to perform server processing

```
void dg_echo(int sockfd, SA *pcliaddr, socklen_t clilen)
{
    int n;
    socklen_t len;
    char mesg[MAXLINE];

    for (;;) {
        len = clilen;

        n = Recvfrom(sockfd, mesg, MAXLINE, 0,
                     pcliaddr, &len);

        Sendto(sockfd, mesg, n, 0,
               pcliaddr, len);
    }
}
```

9.2.2 UDP Echo Client

```
int
main(int argc, char **argv)
{
    int                 sockfd;
    struct sockaddr_in  servaddr;

    if (argc != 2)
        err_quit("usage: tcpcli <IPaddress>");

    sockfd = Socket(AF_INET, SOCK_STREAM, 0);
```

```
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(SERV_PORT);
    Inet_pton(AF_INET, argv[1], &servaddr.sin_addr);

    Connect(sockfd, (SA *) &servaddr, sizeof(servaddr));

    str_cli(stdin, sockfd);        /* do it all */

    exit(0);
}
```

**str_cli Function reads a line of text from standard input, writes it to the server, reads back the server's echo of the line, and outputs the echoed line to standard output.

9.2.3 Code example UDP Echo Server/Client

Server:

```
#define PORT 9877
#define MAXLINE 1024

void dg_echo(int sockfd, struct sockaddr *cliaddr, socklen_t clilen){
    int n;
    socklen_t len;
    char mesg[MAXLINE];

    for( ; ; ){
        len = clilen;

        n = recvfrom(sockfd, mesg, MAXLINE, 0, cliaddr, &len);

        char str[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &((struct sockaddr_in *)cliaddr)->sin_addr, str, sizeof(str));
        printf("Received a datagram from %s:%d\n", str, ntohs(((struct sockaddr_in *)cliaddr)-
>sin_port));

        sendto(sockfd, mesg, n, 0, cliaddr, len);
    }
```

```
}

int main(){
    int sockfd;
    struct sockaddr_in servaddr, cliaddr;

    //creat UDP secket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port        = htons(PORT);

    //Blind IP and Port for Sever
    bind(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));

    printf("UDP Echo Server is running form PORT: %d...\n", PORT);

    dg_echo(sockfd, (struct sockaddr *)&cliaddr, sizeof(cliaddr));

    return 0;
}
```

client:

```
#define PORT 9877
#define SERVER_IP "127.0.0.1"
#define MAXLINE 1024

void dg_cli(FILE *fp, int sockfd, const struct sockaddr *pservaddr, socklen_t servlen) {
    char sendline[MAXLINE], recvline[MAXLINE + 1];
    int n;

    printf("Send data to server (Ctrl+D to Exit):\n");
    while (fgets(sendline, MAXLINE, fp) != NULL) {
        // send datagram to server
        sendto(sockfd, sendline, strlen(sendline), 0, pservaddr, servlen);

        // wait server reply (Blocking I/O)
```

```
        n = recvfrom(sockfd, recvline, MAXLINE, 0, NULL, NULL);

        recvline[n] = 0; // Null-terminate
        fputs(recvline, stdout);
    }
}

int main(int argc, char **argv) {
    int sockfd;
    struct sockaddr_in servaddr;

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &servaddr.sin_addr);

    dg_cli(stdin, sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));

    return 0;
}
```

Output

Terminal 1

```
./udpsrv

UDP Echo Server is running form PORT: 9877...
Received a datagram from 127.0.0.1:58975
Received a datagram from 127.0.0.1:58975
Received a datagram from 127.0.0.1:34219
Received a datagram from 127.0.0.1:34219
```

Terminal 2

```
./udpcli

Send data to server (Ctrl+D to Exit):
hello
```

```
hello
I am Dat
I am Dat
```

Terminal 3

```
./udpcli

Send data to server (Ctrl+D to Exit):
hello
hello
I am Dat
I am Dat
```

9.3 Lost Datagrams

Unreliable: Unlike TCP, which has automatic checking and retransmission mechanisms, UDP does not guarantee that packets will reach their destination. Packets can be dropped by any router along the way.

Consequence:If the request packet sent by the client is lost, or the response packet sent back by the server is lost

-> The client will block permanently at the recvfrom function waiting for a response that will never appear.

Timeout solution: The most common way to prevent the client from hanging is to set a time limit (timeout) for the recvfrom function. If no response arrives within that time, the function will automatically abort.

Major Limitation: A timeout only tells you that an error has occurred, but it cannot determine exactly which packet was lost:

- Was the request packet sent from the client to the server lost?
- Or was the reply packet sent back from the server to the client lost?

9.4 connect() with UDP

No 3-Way Handshake: Unlike TCP, calling connect() for UDP does not create a physical connection or send any packets across the network (there is no handshake at all).

UDP Socket Classification:

1. unconnected UDP socket: the default when we create a UDP socket

```
socket()
bind()

sendto()
recvfrom()
```

For each transmission, specify the destination address:

```
Client
  |
  | sendto(data, server_addr)
  ↓
Server
  |
  | recvfrom(data, client_addr)
  ↓
```

2. connected UDP socket: the result of **calling connect** on a UDP socket

```
socket()
connect()

write()    ||    send()
read()     ||    send()
```

note:

- Does not create a TCP connection and involves no handshake.
- Kernel Notification: It mainly tells the kernel: "From now on, I want this UDP socket to communicate with this specific address."

Calling connect Multiple Times for a UDP Socket

Unlike TCP (which only allows calling connect exactly once in a socket's lifetime), a UDP socket allows you to call connect multiple times for two main purposes:

- Switching to a New Peer (Destination): You can call connect again to redirect the socket to a completely new IP address and port.
- Disconnecting (Unconnecting the Socket): To return the UDP socket back to the "unconnected" state.

Performance Problem: Unconnected vs. Connected Socket

Unconnected UDP Socket: Every time you call sendto, the kernel must perform two lookup operations: bind the address temporarily, unbind it when done

Connected UDP Socket: Because the destination address is permanently saved in the kernel after calling connect, the system bypasses these repetitive steps. You can use standard write or send instead of sendto, resulting in faster transmission when sending multiple packets to the same destination.

9.5 lack of Flow Control

No Flow Control in UDP: The sender can transmit data much faster than the receiver can process it.

Buffer Overflow and Packet Loss: When the receiver's queue fills up, the kernel simply drops incoming datagrams.

Silent Drop: The application is never notified that any packets were lost.

10 Socket Operations: getsockopt/setsockopt, Generic/IPv4/TCP Socket Options, fcntl Function

10.1 getsockopt/setsockopt Function

Ways to get and set the options that affect a socket

```
int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen);


int setsockopt(int sockfd, int level, int optname, const void *optval,
socklen_t optlen);
```

Core Parameters

Parameter	Description
`sockfd`	Must be an open socket file descriptor.
`level`	Specifies which networking layer or protocol handles the socket option, such as the generic socket layer, IPv4, IPv6, TCP, or SCTP.

Parameter	Description
`optval`	Pointer to the variable containing the option value. • **`setsockopt()`**: The system reads the new value from this memory location. • **`getsockopt()`**: The system writes the current option value to this memory location.
`optlen`	Specifies the size of the `optval` variable. • **`setsockopt()`**: Passed by value. • **`getsockopt()`**: Value-result parameter; the caller provides the buffer size, and the system updates it with the actual size.

10.2 Generic/IPv4/TCP Socket Options

10.3 fcntl Function

fcntl (file control): A standard POSIX function used to perform control and configuration operations on file descriptors (including sockets).

```
int fcntl(int fd, int cmd, ... /* int arg */ );
                Returns: depends on cmd if OK, −1 on error
```

funtion:

- Nonblocking I/O (O_NONBLOCK): Switch the socket to non-blocking mode (read/write operations will not block the process if there is no data available).
- Signal-driven I/O (O_ASYNC): Automatically generate a SIGIO signal to notify the process when the socket state changes.

Enable the Nonblocking flag (O_NONBLOCK):

```
int flags = fcntl(fd, F_GETFL, 0); // 1. Get current flags
flags |= O_NONBLOCK;               // 2. Turn on O_NONBLOCK
fcntl(fd, F_SETFL, flags);         // 3. Set flags back
```

The wrong way: Turning on only O_NONBLOCK directly turns off all other flags (like clearing headlights, wipers, and horn), causing other features to break!

The correct 3-step way using fcntl:

- fcntl(fd, F_GETFL, 0): Take a "snapshot" of all current flags/buttons and save them into the flags variable.
- flags |= O_NONBLOCK: Keep all existing flags intact while adding/turning on the non-blocking flag.

- fcntl(fd, F_SETFL, flags): Send the updated configuration back to the system. This preserves all previous settings while successfully adding the non-blocking feature!

Disable the Nonblocking flag:

```
flags &= ~O_NONBLOCK;             // Use bitwise AND with NOT
fcntl(fd, F_SETFL, flags);
```

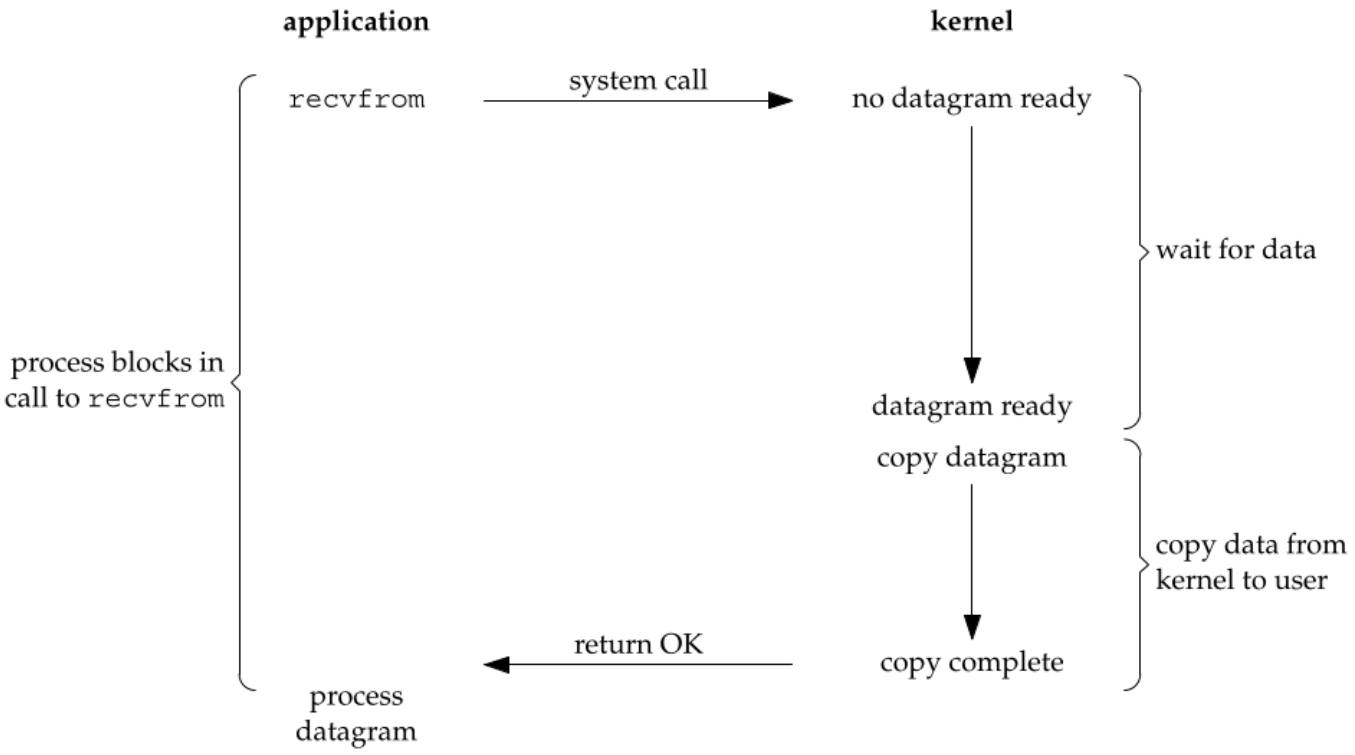
11 I/O Multiplexing: I/O Models, blocking vs. non-blocking I/O, select / poll / pselect Functions

11.1 I/O Models

The two I/O models:

- blocking I/O
- nonblocking I/O
- I/O multiplexing (select and poll)

1.**blocking I/O**



alt text

Process State: When a process calls the recvfrom() function but the kernel has not yet received any data (datagram) from the network, the kernel puts that process into a sleeping/blocked state.

CPU Resources: The process is removed from the CPU run queue (CPU scheduler), completely freeing up the CPU for other processes. The process consumes zero CPU resources while waiting.

Waking Up: When a packet arrives at the Network Interface Card (NIC), a hardware interrupt occurs. The kernel handles it, copies the data into the socket's receive buffer, and then wakes the process up to continue executing the next instruction.

2.**nonblocking I/O**

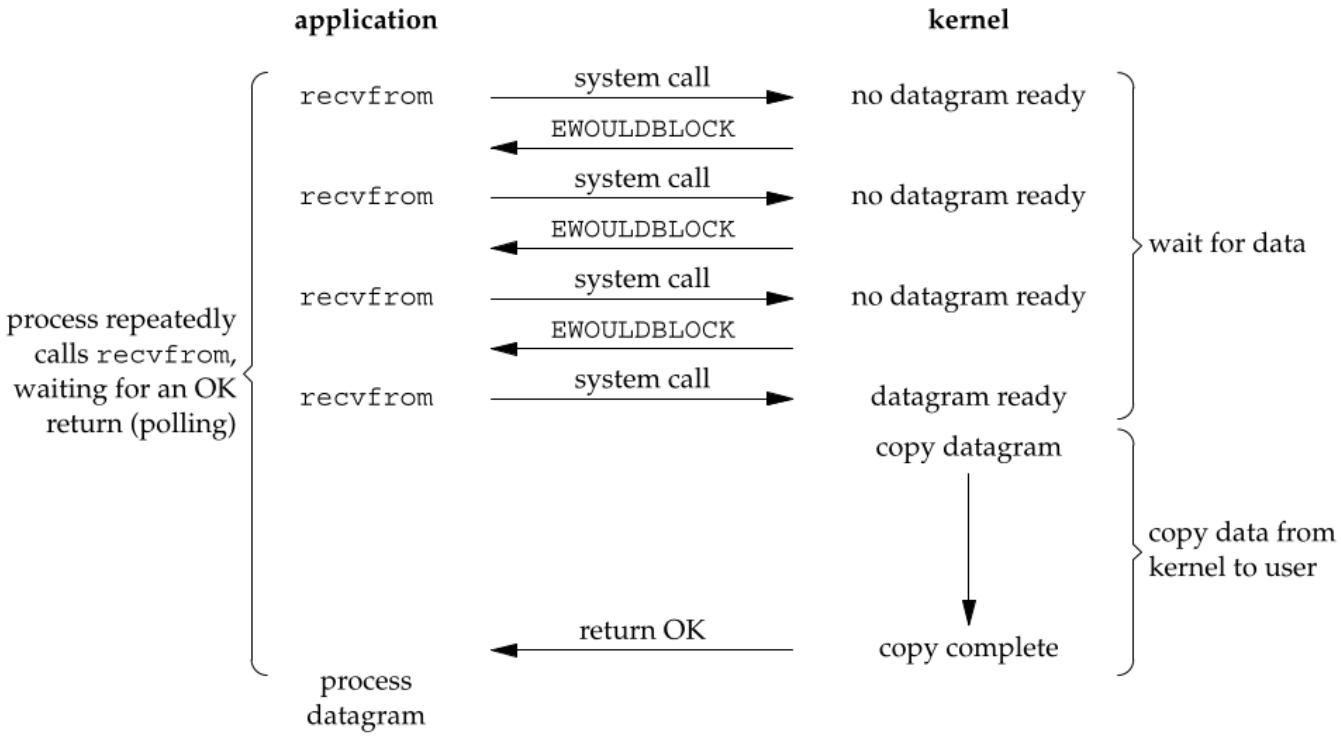


Figure 6.2 Nonblocking I/O model.

alt text

Process State: When calling recvfrom(), if the kernel does not have data ready yet, the function returns immediately the error code EWOULDBLOCK. The process is not put into a sleeping state and continues executing the next instruction.

Polling Mechanism: to know when data is available, your program must continuously call (loop/poll) the recvfrom() function until it successfully receives data.

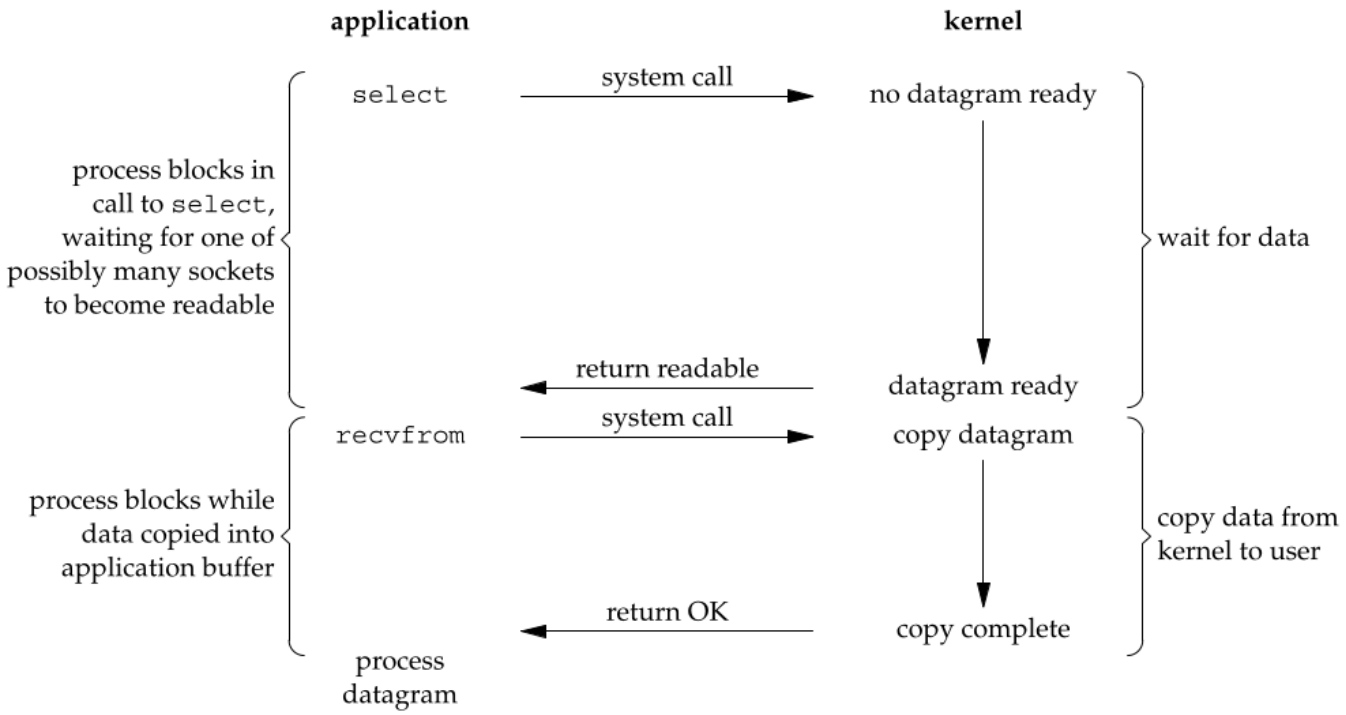
Disadvantage of Pure Polling: Constantly making system calls and checking wastes a lot of CPU resources because the CPU has to work continuously just to keep asking the kernel, "Is there any data yet?"

Note: This is often a waste of CPU time, but this model is occasionally encountered, normally on systems dedicated to one function

---

3.**I/O multiplexing (select and poll)**

With I/O multiplexing, we call select or poll and block in one of these two system calls, instead of blocking in the actual I/O system call



We block in a call to select, waiting for the datagram socket to be readable. When select returns that the socket is readable, we then call recvfrom to copy the datagram into our application buffer.

There does not appear to be any advantage, and in fact, there is a slight disadvantage because using select requires two system calls instead of one. But the advantage in using select, is that we can wait for more than one descriptor to be ready.

11.2 select Funtion

This function allows the process to instruct the kernel to wait for any one of multiple events to occur and to wake up the process only when one or more of these events occurs or when a specified amount of time has passed.

As an example, we can call select and tell the kernel to return only when:

- Any of the descriptors in the set {1, 4, 5} are ready for reading
- Any of the descriptors in the set {2, 7} are ready for writing
- Any of the descriptors in the set {1, 4} have an exception condition pending 10.2 seconds have elapsed

```
int select(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset, const struct timeval *timeout);

Returns: positive count of ready descriptors, 0 on timeout, −1 on error
```

Timeout Type	`timeout` Value	Description
Infinite Wait	`NULL`	The process blocks until at least one descriptor becomes ready.
Timed Wait	`struct timeval`	Waits for the specified time using `tv_sec` (seconds) and `tv_usec` (microseconds).
No Wait (Polling)	`tv_sec = 0`, `tv_usec = 0`	Checks the descriptors immediately and returns without blocking.

```
void FD_ZERO(fd_set *fdset); // Clear all bits in fdset

void FD_SET(int fd, fd_set *fdset); // Turn on the bit for fd in fdset

void FD_CLR(int fd, fd_set *fdset); // Turn off the bit for fd in fdset

int FD_ISSET(int fd, fd_set *fdset); // Check if the bit for fd is on in fdset
```

maxfdp1 Parameter (Common Pitfall #1):

```
Meaning: Stands for Max File Descriptor Plus 1.

Example: If you are monitoring file descriptors 1, 4, and 5, the highest value is 5. Therefore, you must pass maxfdp1 = 6 (since indexing starts at 0, the total range to scan includes 6 elements: 0 through 5).

Purpose: To optimize kernel performance. Instead of forcing the kernel to scan through all 1,024 default file descriptors (which is inefficient), passing maxfdp1 restricts the kernel's scan range from 0 up to that specific limit.
```

select() as a "Value-Result" Parameter (Common Pitfall #2):

```
Behavior: When you call select(), you pass in set structures (fd_set) to tell the kernel which descriptors you want to monitor. However, when select() returns, it overwrites those exact fd_set variables!

Bit Modification: The kernel automatically clears (0) the bits for any sockets that have no
```

```
activity, leaving only the bits for sockets that are actually ready.

Consequence: Because select() modifies (destroys) the bitmasks in place, you must re-populate or re-enable the file descriptors using macros like FD_SET at the start of every loop iteration. If you forget, subsequent calls to select() will monitor an empty set!
```

11.3 poll Funtions

```
int poll(struct pollfd *fdarray, unsigned long nfds, int timeout);

Returns: count of ready descriptors, 0 on timeout, –1 on error
```

struct pollfd Structure: Unlike select(), which uses a single shared bitmask (value-result) that gets overwritten, poll() clearly separates input conditions from output results for each descriptor using the following structure

```
struct pollfd {
    int   fd;         // File descriptor to monitor
    short events;     // Requested events (input conditions to watch for)
    short revents;    // Returned events (actual results set by kernel)
};
```

Timeout Value	Description
`INFTIM`	Wait forever.
`0`	Return immediately; do not block.
`> 0`	Wait for the specified number of milliseconds.

Major Advantages of poll over select:

```
No Hard Size Limit (FD_SETSIZE): With select(), you are constrained by the fixed size of fd_set (typically 1024). In contrast, with poll(), you can dynamically allocate an array of pollfd structures of any custom size and simply pass the count (nfds) to the kernel.

Cleaner State Management: Separating requested events (events) and returned events (revents) eliminates the troublesome value-result overwrites seen in select(), avoiding common logic bugs and sparing you from having to reinitialize your input masks on every loop iteration.
```

11.4 pselect Function

```
int pselect(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset, const struct
timespec *timeout, const sigset_t *sigmask);

Returns: count of ready descriptors, 0 on timeout, –1 on error
```

Solving Race Conditions with pselect

```
if (intr_flag)
    handle_intr();

//racecondition if signal appear here

if ((nready = select(...)) < 0) {
    if (errno == EINTR) {
        if (intr_flag)
            handle_intr();
    }
}
```

The problem is that between the test of intr_flag and the call to select, if the signal occurs, it will be lost if select blocks forever

Fix by pselect

```
sigset_t newmask, oldmask, zeromask;

sigemptyset(&zeromask);
sigemptyset(&newmask);
sigaddset(&newmask, SIGINT);
sigprocmask(SIG_BLOCK, &newmask, &oldmask); /* block SIGINT */

if (intr_flag)
    handle_intr();

/* handle the signal */
if ((nready = pselect(..., &zeromask)) < 0) {
    if (errno == EINTR) {
        if (intr_flag)
            handle_intr();
    }
    ...
```

```
}
```

Proactive Signal Blocking: The program explicitly blocks signals (such as SIGINT) using sigprocmask before checking your flags.

Atomic Operation: When calling pselect, you pass in a temporary signal mask (often a blank mask). The system atomically swaps the mask, checks the file descriptors, and enters the sleep state without any gaps.

Automatic Restoration: The moment pselect returns, the process's original signal mask is automatically restored, completely eliminating the vulnerability window where incoming signals could otherwise be missed.

11.5 Build a chat-room server using select() / poll() and compare

11.5.1 select()

Server:

```
#define PORT 9877
#define MAXLINE 1024
#define FD_SETSIZE_CUSTOM 1024

int main(){
    int i, maxi, maxfd, listenfd, connfd, sockfd;
    int nready, client[FD_SETSIZE_CUSTOM];

    ssize_t n;
    fd_set rset, allset;

    char buf[MAXLINE];
    socklen_t clilen;
    struct sockaddr_in cliaddr, servaddr;

     // 1. Create the TCP Listening socket
    listenfd = socket(AF_INET, SOCK_STREAM, 0);

    // Allow address reuse to avoid the "Address already in use" error
    int opt = 1;
    setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

```
bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family      = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port        = htons(PORT);

bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
listen(listenfd, 5);

maxfd = listenfd;           // Initialize maxfd
maxi = -1;                  // No clients in the client[] array yet
for (i = 0; i < FD_SETSIZE_CUSTOM; i++)
    client[i] = -1;         // -1 means an empty slot

FD_ZERO(&allset);
FD_SET(listenfd, &allset);

printf("Chat-Room Server (select) is running on port %d...\n", PORT);



// 2. Check for new incoming client connections
for ( ; ; ) {
    rset = allset;          // Re-assign because select is value-result
    nready = select(maxfd + 1, &rset, NULL, NULL, NULL);

    if (FD_ISSET(listenfd, &rset)) {
        clilen = sizeof(cliaddr);
        connfd = accept(listenfd, (struct sockaddr *)&cliaddr, &clilen);

        // Find an empty slot in the client[] array
        for (i = 0; i < FD_SETSIZE_CUSTOM; i++) {
            if (client[i] < 0) {
                client[i] = connfd; // Store the client descriptor
                break;
            }
        }

        if (i == FD_SETSIZE_CUSTOM) {
            fprintf(stderr, "Too many clients!\n");
            close(connfd);
        } else {
```

```
                char ipstr[INET_ADDRSTRLEN];
                inet_ntop(AF_INET, &cliaddr.sin_addr, ipstr, sizeof(ipstr));
                printf("New client connected from %s:%d (slot %d)\n", ipstr,
ntohs(cliaddr.sin_port), i);

                FD_SET(connfd, &allset); // Add to the monitored set
                if (connfd > maxfd)
                    maxfd = connfd;      // Update maxfd
                if (i > maxi)
                    maxi = i;            // Update the maximum index in the array
            }

            if (--nready <= 0)
                continue; // No other events, return to the loop
        }

        // 3. Check for incoming data from existing clients
        for (i = 0; i <= maxi; i++) {
            if ((sockfd = client[i]) < 0)
                continue;

            if (FD_ISSET(sockfd, &rset)) {
                // Read data from the client
                if ((n = read(sockfd, buf, MAXLINE)) <= 0) {
                    // Client disconnected or encountered an error
                    getpeername(sockfd, (struct sockaddr *)&cliaddr, &clilen);
                    char ipstr[INET_ADDRSTRLEN];
                    inet_ntop(AF_INET, &cliaddr.sin_addr, ipstr, sizeof(ipstr));
                    printf("Client %s:%d disconnected.\n", ipstr, ntohs(cliaddr.sin_port));

                    close(sockfd);
                    FD_CLR(sockfd, &allset); // Remove from the monitored set
                    client[i] = -1;          // Free the slot
                } else {
                    // Broadcast the received message to ALL other clients
                    buf[n] = 0;
                    printf("Received from client [slot %d]: %s", i, buf);

                    // Send the message to all connected clients
                    for (int j = 0; j <= maxi; j++) {
```

```
                        if (client[j] != -1) {
                            // Format the message with the slot number for distinction
                            char msg_out[MAXLINE + 32];
                            snprintf(msg_out, sizeof(msg_out), "User[%d]: %s", i, buf);
                            write(client[j], msg_out, strlen(msg_out));
                        }
                    }
                }

                if (--nready <= 0)
                    break; // Processed all ready descriptors
            }
        }
    }
}
```

client:

```
#define PORT 9877
#define SERVER_IP "127.0.0.1"
#define MAXLINE 1024

int main(int argc, char **argv) {
    int sockfd;
    struct sockaddr_in servaddr;
    fd_set rset;
    char sendline[MAXLINE], recvline[MAXLINE];
    int maxfdp1;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &servaddr.sin_addr);

    connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
    printf("Connected to the chat room! Enter your message and press Enter:\n");

    FD_ZERO(&rset);
```

```
    maxfdp1 = (sockfd > STDIN_FILENO ? sockfd : STDIN_FILENO) + 1;

    for ( ; ; ) {
        FD_SET(STDIN_FILENO, &rset);
        FD_SET(sockfd, &rset);

        select(maxfdp1, &rset, NULL, NULL, NULL);

        // Case 1: Data available from keyboard input
        if (FD_ISSET(STDIN_FILENO, &rset)) {
            if (fgets(sendline, MAXLINE, stdin) == NULL)
                break; // EOF (Ctrl+D)
            write(sockfd, sendline, strlen(sendline));
        }

        // Case 2: Broadcast message received back from the server
        if (FD_ISSET(sockfd, &rset)) {
            int n = read(sockfd, recvline, MAXLINE);
            if (n <= 0) {
                printf("Server disconnected.\n");
                break;
            }
            recvline[n] = 0;
            fputs(recvline, stdout);
        }
    }

    close(sockfd);
    exit(0);
}
```

Output

terminal 1:

```
./chat_select_server

Chat-Room Server (select) is running on port 9877...
New client connected from 127.0.0.1:42792 (slot 0)
New client connected from 127.0.0.1:42802 (slot 1)
Received from client [slot 0]: hello
```

```
Received from client [slot 1]: hi
Received from client [slot 0]: how are you?
Received from client [slot 1]: i'm fine
Client 127.0.0.1:42802 disconnected.
Client 127.0.0.1:42792 disconnected.
```

terminal 2:

```
./chat_client

Connected to the chat room! Enter your message and press Enter:
hello
User[0]: hello
User[1]: hi
how are you?
User[0]: how are you?
User[1]: i'm fine
```

terminal 3:

```
./chat_client

Connected to the chat room! Enter your message and press Enter:
User[0]: hello
hi
User[1]: hi
User[0]: how are you?
i'm fine
User[1]: i'm fine
```

11.5.2 poll()

server:

```
#define PORT 9877
#define MAXLINE 1024
#define OPEN_MAX 1024  // Maximum number of clients managed in the pollfd array

int main(int argc, char **argv) {
    int i, maxi, listenfd, connfd, sockfd;
    int nready;
```

```
ssize_t n;
char buf[MAXLINE];
socklen_t clilen;
struct sockaddr_in cliaddr, servaddr;
struct pollfd client[OPEN_MAX];

// 1. Create the TCP Listening socket
listenfd = socket(AF_INET, SOCK_STREAM, 0);

int opt = 1;
setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family      = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port        = htons(PORT);

bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
listen(listenfd, 5);

// Initialize the pollfd array
client[0].fd = listenfd;
client[0].events = POLLIN;       // Listen for read events (new connections)
for (i = 1; i < OPEN_MAX; i++)
    client[i].fd = -1;           // Assign -1 to indicate an empty slot

maxi = 0;                        // Index of the largest active element in the array

printf("Chat-Room Server (poll) is running on port %d...\n", PORT);

for ( ; ; ) {
    // Call poll() to wait for events — no complex bit resetting required unlike select
    nready = poll(client, maxi + 1, -1);

    if (nready < 0) {
        perror("poll error");
        exit(1);
    }

    // 2. Check for new connections on the listening socket (client[0])
```

```
        if (client[0].revents & POLLIN) {
            clilen = sizeof(cliaddr);
            connfd = accept(listenfd, (struct sockaddr *)&cliaddr, &clilen);

            // Find an empty slot in the client[] array
            for (i = 1; i < OPEN_MAX; i++) {
                if (client[i].fd < 0) {
                    client[i].fd = connfd;
                    break;
                }
            }

            if (i == OPEN_MAX) {
                fprintf(stderr, "Too many clients!\n");
                close(connfd);
            } else {
                char ipstr[INET_ADDRSTRLEN];
                inet_ntop(AF_INET, &cliaddr.sin_addr, ipstr, sizeof(ipstr));
                printf("New client connected from %s:%d (poll index slot %d)\n", ipstr,
ntohs(cliaddr.sin_port), i);

                client[i].events = POLLIN; // Register interest in read events for the new
client
                if (i > maxi)
                    maxi = i;              // Update the maximum index
            }

            if (--nready <= 0)
                continue; // No other events, return to the loop
        }

        // 3. Check for incoming data from existing clients
        for (i = 1; i <= maxi; i++) {
            if ((sockfd = client[i].fd) < 0)
                continue;

            // Check if readable data or an error/disconnection event has occurred
            if (client[i].revents & (POLLIN | POLLERR | POLLHUP)) {
                if ((n = read(sockfd, buf, MAXLINE)) <= 0) {
                    // Client disconnected or encountered an error
```

```
                    getpeername(sockfd, (struct sockaddr *)&cliaddr, &clilen);
                    char ipstr[INET_ADDRSTRLEN];
                    inet_ntop(AF_INET, &cliaddr.sin_addr, ipstr, sizeof(ipstr));
                    printf("Client %s:%d disconnected.\n", ipstr, ntohs(cliaddr.sin_port));

                    close(sockfd);
                    client[i].fd = -1;  // Simply assign a negative value so the kernel
ignores it in the next poll
                } else {
                    buf[n] = 0;
                    printf("Received from client [poll slot %d]: %s", i, buf);

                    // Broadcast the message to all other clients in the chat room
                    for (int j = 1; j <= maxi; j++) {
                        if (client[j].fd != -1) {
                            char msg_out[MAXLINE + 32];
                            snprintf(msg_out, sizeof(msg_out), "User[%d]: %s", i, buf);
                            write(client[j].fd, msg_out, strlen(msg_out));
                        }
                    }
                }

                if (--nready <= 0)
                    break; // Finished processing all events
            }
        }
    }
    return 0;
}
```

client remains unchanged

Output is the same

terminal 1:

```
./chat_poll_server

Chat-Room Server (poll) is running on port 9877...
New client connected from 127.0.0.1:40486 (poll index slot 1)
New client connected from 127.0.0.1:54884 (poll index slot 2)
```

```
Received from client [poll slot 1]: hello
Received from client [poll slot 2]: hi
Received from client [poll slot 1]: How are you?
Received from client [poll slot 2]: I'm fine
Client 127.0.0.1:54884 disconnected.
Client 127.0.0.1:40486 disconnected.
```

terminal 2:

```
./chat_client

Connected to the chat room! Enter your message and press Enter:
hello
User[1]: hello
User[2]: hi
How are you?
User[1]: How are you?
User[2]: I'm fine
```

terminal 3:

```
./chat_client

Connected to the chat room! Enter your message and press Enter:
User[1]: hello
hi
User[2]: hi
User[1]: How are you?
I'm fine
User[2]: I'm fine
```

11.5.3 Compare select and poll

Comparison Criteria	`chat_select_server` Version (`select`)	`chat_poll_server` Version (`poll`)
Storage Structure	Uses an integer array `client[]` to store individual sockets, combined with the `fd_set allset` bitmask for management.	Directly uses an array of `struct pollfd client[OPEN_MAX]` structures.
Initialization Before Loop	Requires `FD_ZERO` and `FD_SET` to turn on bits for the listening socket.	Only requires assigning `client[0].fd = listenfd` and `client[0].events = POLLIN`.

Comparison Criteria	`chat_select_server` Version (`select`)	`chat_poll_server` Version (`poll`)
Each Event Loop	Requires `rset = allset` because `select()` modifies the `fd_set` and removes unready descriptors.	Requires no resetting; call `poll(client, maxi + 1, -1)` directly, and the kernel fills results into the `revents` field.
Handling Disconnections	Requires calling `FD_CLR(sockfd, &allset)` to manually clear the descriptor from the `fd_set`.	Assign `client[i].fd = -1`; `poll()` ignores that slot in subsequent scans.
Scalability	Limited by `FD_SETSIZE` (typically 1024), making it less suitable for a large number of file descriptors.	No equivalent `FD_SETSIZE` limit; scaling is mainly limited by system file descriptor resources and memory.
Handling Large Descriptors	Requires calculating `maxfdp1` (largest descriptor + 1). The `select()` implementation scans the descriptor range up to this value.	The actual numeric value of `fd` does not determine the number of entries scanned; `poll()` scans the `nfds` entries in the array.
Timeout Precision	Uses `struct timeval`, with seconds (`tv_sec`) and microseconds (`tv_usec`).	Uses an integer `timeout`, with the timeout specified in milliseconds.

12 epoll: interest list vs. ready list, level-triggered vs. edge-triggered, epoll_create/epoll_ctl/epoll_wait, why epoll scales better than select/poll

Key advantages of epoll:

- Scalability: epoll performs much better than select() and poll() when handling a large number of file descriptors.
- Flexible notification modes: Supports both level-triggered and edge-triggered modes. (While select() and poll() only support level-triggered, signal-driven I/O only supports edge-triggered).

12.1 interest list vs. ready list, level-triggered vs. edge-triggered

Epoll Instance: This structure maintains two important lists

- Interest list: The list of file descriptors that the process registers to monitor.
- Ready list: The subset list of file descriptors that are ready for I/O.



---

level-triggered vs. edge-triggered

Level-Triggered (Default):

- How it works: The OS keeps notifying you of an event (e.g., socket is readable) as long as that state still exists.
- Example: If the socket buffer has 100 bytes of data and you only read 40 bytes, the next time you call epoll_wait, the kernel will still report that this socket has data to read because the remaining 60 bytes are still in the buffer.

Edge-Triggered (Performance-optimized):

- How it works: The OS only notifies you once right when the state suddenly changes (from "no data" to "new data arrived").
- Example: If new data arrives, the kernel reports it once. If you only read part of it and leave some in the buffer, the kernel will stay quiet on subsequent calls unless a brand-new packet arrives.

Note:

Non-blocking I/O with Edge-Triggered: When programming with Edge-Triggered mode, you must use a loop (while) combined with a non-blocking socket to read continuously until the read() function returns an EAGAIN or EWOULDBLOCK error.

- The EAGAIN error is how the kernel politely tells you: "The buffer is completely out of data, you can stop the read loop now!".

Compare level-triggered vs. edge-triggered

Criteria	Level-Triggered (LT)	Edge-Triggered (ET)
Notification Frequency	Keeps notifying while the event is still not handled.	Notifies only once when the state changes.
Buffer Handling	You do not have to read the entire buffer in one call.	You must read the buffer until `EAGAIN` .
Non-blocking Requirement	Not required, but recommended.	The socket must be configured as non-blocking.
Code Complexity	Simple and easy to debug.	More complex; easy to miss events if the code is not careful.
System Performance	Good for normal applications.	Very efficient for large-scale systems, such as C10K.

12.2 epoll_create/epoll_ctl/epoll_wait

This API consists of 3 main functions:

- epoll_create(): Creates a new epoll instance and returns a file descriptor for it.
- epoll_ctl(): Adds, removes, or modifies file descriptors and event masks in the interest list.
- epoll_wait(): Waits for and returns file descriptors that are ready for I/O from the ready list.

12.2.1 epoll_create

```
int epoll_create(int size);
            Returns file descriptor on success, or –1 on error
```

size: specifies the number of file descriptors that we expect to monitor via the epoll instance

epoll_create1() (Modern API since Linux 2.6.27):

- Improvement: Replaces the obsolete size parameter with a flags parameter for custom behavior.
- Key Flag (EPOLL_CLOEXEC): Instructs the kernel to automatically set the FD_CLOEXEC flag on the new file descriptor, preventing FD leaks to child processes when calling exec().

12.2.2 epoll_clt

The epoll_ctl() system call modifies the interest list of the epoll instance referred to by the file descriptor epfd.

```
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *ev);
                    Returns 0 on success, or –1 on error
```

- epfd: File descriptor representing the epoll instance.

- fd: File descriptor to operate on.
- op: Operation to perform (EPOLL_CTL_ADD, EPOLL_CTL_MOD, or EPOLL_CTL_DEL).

```
EPOLL_CTL_ADD: Adds an fd to the interest list along with the event configuration in ev. Returns an EEXIST error if the fd already exists in the list.

EPOLL_CTL_MOD: Modifies the event configuration for an existing fd. Returns an ENOENT error if the fd is not in the list.

EPOLL_CTL_DEL: Removes an fd from the list (the ev parameter is ignored). Returns an ENOENT error if the fd does not exist in the list.
```

- ev: Pointer to the configuration structure specifying the events of interest (struct epoll_event).

```
struct epoll_event {
    uint32_t     events;   // epoll events (bit mask)
    epoll_data_t data;     // User data
};

typedef union epoll_data {
    void     *ptr;   // Pointer to user-defined data
    int       fd;    // File descriptor
    uint32_t  u32;   // 32-bit integer
    uint64_t  u64;   // 64-bit integer
} epoll_data_t;

=>

epoll_event
├── events
│   └── Bit mask: EPOLLIN, EPOLLOUT, ...
│
└── data
    └── epoll_data_t (union)
        ├── ptr
        ├── fd
        ├── u32
        └── u64
```

Using epoll_create() and epoll_ctl()

```
int epfd;
struct epoll_event ev;

epfd = epoll_create(5);
if (epfd == -1)
    errExit("epoll_create");

ev.data.fd = fd;
ev.events = EPOLLIN;
if (epoll_ctl(epfd, EPOLL_CTL_ADD, fd, ev) == -1)
    errExit("epoll_ctl");
```

max_user_watches:

Linux sets a limit on the total number of file descriptors that each user is allowed to register across all epoll instances.

```
cat /proc/sys/fs/epoll/max_user_watches
3538142
```

12.2.3 epoll_wait (Waiting for Events:)

System call returns information about ready file descriptors from the epoll instance referred to by the file descriptor epfd

```
int epoll_wait(int epfd, struct epoll_event *evlist, int maxevents, int timeout);
        Returns number of ready file descriptors, 0 on timeout, or –1 on error
```

How to identify the File Descriptor from an event:

- The data field in the struct epoll_event (such as ev.data.fd or ev.data.ptr) is the sole mechanism to determine which file descriptor corresponds to the returned event.
- Therefore, when calling epoll_ctl() to add an entry to the interest list, the programmer must explicitly assign a value to the data field beforehand.

```
struct epoll_event ev;
ev.events = EPOLLIN;     // Monitor read events
ev.data.fd = 5;          // Assign fd = 5 here to remember it for later!

if (epoll_ctl(epfd, EPOLL_CTL_ADD, 5, &ev) == -1) {
    perror("epoll_ctl: add");
    // Handle error
```

```
}
```

Timeout modes in epoll_wait:

- -1: Block indefinitely until an event occurs or it is interrupted by a signal.
- 0: Check immediately (non-blocking) to see if any events are ready right away, returning instantly whether there are events or not.
- if > 0: Block for a maximum specified time period in milliseconds.

Special feature in multithreaded programs:

- One thread can use epoll_ctl() to add a new file descriptor to the interest list while another thread is blocking and waiting in epoll_wait().
- The system will update immediately, and epoll_wait() will return the readiness information of those newly added file descriptors.

12.2.4 Why epoll scales better than select/poll

Data transfer cost between User Space and Kernel Space:

- select() and poll(): On every call, the program must force the kernel to scan through the entire list of N file descriptors. At the same time, you have to copy that entire list from user space down to kernel space, and then the kernel copies back the status of all file descriptors.
- epoll(): You only use epoll_ctl() to register the monitoring list once into kernel space. In subsequent epoll_wait() calls, you do not need to pass any file descriptor list information back down, and the kernel only returns the file descriptors that actually have ready events.

Table 63-9: Times taken by *poll()*, *select()*, and *epoll* for 100,000 monitoring operations

Number of descriptors monitored (*N*)	*poll()* CPU time (seconds)	*select()* CPU time (seconds)	*epoll* CPU time (seconds)
10	0.61	0.73	0.41
100	2.9	3.0	0.42
1000	35	35	0.53
10000	990	930	0.66

12.3 Build the chat-room server once more using epoll (edge-triggered, non-blocking sockets)

server:

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <sys/epoll.h>

#define PORT 9877
#define MAX_EVENTS 1024
#define BUFFER_SIZE 1024
#define MAX_CLIENTS 10000

// List to manage connected clients
int client_fds[MAX_CLIENTS];
int client_count = 0;

// Function to add a client to the management list
void add_client(int fd) {
    if (client_count < MAX_CLIENTS) {
        client_fds[client_count++] = fd;
    }
}

// Function to remove a client from the management list
void remove_client(int fd) {
    for (int i = 0; i < client_count; i++) {
        if (client_fds[i] == fd) {
            client_fds[i] = client_fds[client_count - 1];
            client_count--;
            break;
        }
    }
}

// Function to broadcast a message to all clients EXCEPT the sender (sender_fd)
void broadcast_message(int sender_fd, const char *msg, ssize_t len) {
    for (int i = 0; i < client_count; i++) {
        int target_fd = client_fds[i];
        if (target_fd != sender_fd) {
```

```
            // Send data to other clients
            write(target_fd, msg, len);
        }
    }
}

// Function to configure a socket to Non-blocking mode
int set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags == -1) return -1;
    flags |= O_NONBLOCK;
    return fcntl(fd, F_SETFL, flags);
}

int main(){
    int server_fd, epoll_fd;
    struct sockaddr_in address;
    struct epoll_event ev, events[MAX_EVENTS];

    // 1. Create server socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Allow port reuse immediately after restart
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    set_nonblocking(server_fd);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }
```

```
    if (listen(server_fd, SOMAXCONN) < 0) {
        perror("listen failed");
        exit(EXIT_FAILURE);
    }

    // 3. Create epoll instance using epoll_create1
    epoll_fd = epoll_create1(0);
    if (epoll_fd == -1) {
        perror("epoll_create1 failed");
        exit(EXIT_FAILURE);
    }

    // 4. Register server_fd into epoll with Edge-Triggered mode (EPOLLET)
    ev.events = EPOLLIN | EPOLLET;
    ev.data.fd = server_fd;

    if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, server_fd, &ev) == -1) {
        perror("epoll_ctl server_fd failed");
        exit(EXIT_FAILURE);
    }

    printf("Chat Room Server (epoll ET + Non-blocking) is running on port %d...\n", PORT);

    while(1){
        int nfds = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
        if (nfds == -1) {
            if (errno == EINTR) continue;
            perror("epoll_wait failed");
            break;
        }

        for (int i = 0; i < nfds; i++){
            if (events[i].data.fd == server_fd){
                // --- HANDLE NEW CONNECTIONS (MUST USE A LOOP DUE TO ET MODE) ---
                while(1){
                    struct sockaddr_in client_addr;
                    socklen_t client_len = sizeof(client_addr);
                    int client_fd = accept(server_fd, (struct sockaddr *)&client_addr,
&client_len);
```

```
            if (client_fd == -1) {
                if (errno == EAGAIN || errno == EWOULDBLOCK) {
                    // Accepted all pending connections in the queue
                    break;
                } else {
                    perror("accept failed");
                    break;
                }
            }

            // Switch client socket to non-blocking
            set_nonblocking(client_fd);

            // Register client_fd into epoll with EPOLLIN, EPOLLET, and EPOLLRDHUP
            ev.events = EPOLLIN | EPOLLET | EPOLLRDHUP;
            ev.data.fd = client_fd;
            if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, client_fd, &ev) == -1) {
                perror("epoll_ctl client_fd failed");
                close(client_fd);
                continue;
            }

            add_client(client_fd);
            printf("-> New client connected! (fd: %d)\n", client_fd);
        }
    }else{
        int client_fd = events[i].data.fd;

        // Check if the client disconnected
        if (events[i].events & (EPOLLRDHUP | EPOLLHUP | EPOLLERR)) {
            printf("<- Client (fd: %d) disconnected.\n", client_fd);
            remove_client(client_fd);
            epoll_ctl(epoll_fd, EPOLL_CTL_DEL, client_fd, NULL);
            close(client_fd);
            continue;
        }

        // Due to Edge-Triggered mode, must read the buffer CLEANLY in a loop
        char buffer[BUFFER_SIZE];
```

```
                int connection_closed = 0;

                while(1){
                    memset(buffer, 0, BUFFER_SIZE);
                    ssize_t count = read(client_fd, buffer, sizeof(buffer) - 1);
                    if(count == -1){
                        if (errno == EAGAIN || errno == EWOULDBLOCK){
                            break;
                        }
                        else{
                            perror("read failed");
                            connection_closed = 1;
                            break;
                        }
                    }else if(count == 0){
                        // Client closed connection normally
                        connection_closed = 1;
                        break;
                    }else{
                        // Print to server screen
                        printf("[Client %d]: %s", client_fd, buffer);

                        // Prepare formatted message to broadcast to other clients
                        char broadcast_buf[BUFFER_SIZE + 64];
                        int msg_len = snprintf(broadcast_buf, sizeof(broadcast_buf), "Client
[%d]: %s", client_fd, buffer);

                        // Send message to all other clients in the chat room
                        broadcast_message(client_fd, broadcast_buf, msg_len);
                    }
                }

                if(connection_closed){
                    printf("<- Closing client connection (fd: %d)\n", client_fd);
                    remove_client(client_fd);
                    epoll_ctl(epoll_fd, EPOLL_CTL_DEL, client_fd, NULL);
                    close(client_fd);
                }
            }
        }
```

```
    }

    close(server_fd);
    close(epoll_fd);

    return 0;
}
```

client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/select.h>

#define PORT 9877
#define SERVER_IP "127.0.0.1"
#define MAXLINE 1024

int main(int argc, char **argv) {
    int sockfd;
    struct sockaddr_in servaddr;
    fd_set rset;
    char sendline[MAXLINE], recvline[MAXLINE];
    int maxfdp1;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &servaddr.sin_addr);

    connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
    printf("Connected to the chat room! Enter your message and press Enter:\n");
```

```
    FD_ZERO(&rset);
    maxfdp1 = (sockfd > STDIN_FILENO ? sockfd : STDIN_FILENO) + 1;

    for ( ; ; ) {
        FD_SET(STDIN_FILENO, &rset);
        FD_SET(sockfd, &rset);

        select(maxfdp1, &rset, NULL, NULL, NULL);

        // Case 1: Data available from keyboard input
        if (FD_ISSET(STDIN_FILENO, &rset)) {
            if (fgets(sendline, MAXLINE, stdin) == NULL)
                break; // EOF (Ctrl+D)
            write(sockfd, sendline, strlen(sendline));
        }

        // Case 2: Broadcast message received back from the server
        if (FD_ISSET(sockfd, &rset)) {
            int n = read(sockfd, recvline, MAXLINE);
            if (n <= 0) {
                printf("Server disconnected.\n");
                break;
            }
            recvline[n] = 0;
            fputs(recvline, stdout);
        }
    }

    close(sockfd);
    exit(0);
}
```

Output

terminal 1:

```
./chat_epoll_server

Chat Room Server (epoll ET + Non-blocking) is running on port 9877...
-> New client connected! (fd: 5)
-> New client connected! (fd: 6)
```

```
[Client 5]: hello
[Client 6]: hi
[Client 5]: how are you?
[Client 6]: find!
[Client 6]: what about you?
[Client 5]: good
<- Client (fd: 5) disconnected.
<- Client (fd: 6) disconnected.
```

terminal 2:

```
./chat_client

Connected to the chat room! Enter your message and press Enter:
hello
Client [6]: hi
how are you?
Client [6]: find!
Client [6]: what about you?
good
```

terminal 3:

```
./chat_client
Connected to the chat room! Enter your message and press Enter:
Client [5]: hello
hi
Client [5]: how are you?
find!
what about you?
Client [5]: good
```

Why we cannot use the Interest List to broadcast and have to use client_fds[MAX_CLIENTS] instead:

The Linux kernel hides the "Interest List" from User Space:

- Although the kernel maintains the Interest List internally, Linux does not provide any API function for user space to query or look up this list (for instance, a function like epoll_get_all_fds(epoll_fd) does not exist).

No "peeking" allowed:

- Once a client_fd is added to epoll, it lives deep inside the kernel space. Your application can interact with it through system calls (like epoll_ctl or epoll_wait), but it cannot "peek inside" to list or extract all the file descriptors currently being monitored. Therefore, maintaining a separate user-space array (like client_fds) is necessary to keep track of active connections for tasks like broadcasting.

13 Combining multiplexing with threads: single-thread + select/poll/epoll vs. multi-thread + select/poll/epoll designs, and when each is appropriate

13.1 single-thread + select/poll/epoll

Single-Thread + select/poll Model Mechanics:

A single process or thread can monitor a large batch of file descriptors (sockets) concurrently, thanks to kernel support (select/poll).

When select/poll reports that certain FDs are ready, the thread iterates through a loop to handle each FD one by one sequentially (finishing one before starting the next), rather than processing multiple FDs truly in parallel at the exact same instant.

```
                  ┌──────────────────┐
Client A ────────>│                  │
Client B ────────>│  poll()          │
Client C ────────>│      ↓           │
Client D ────────>│process each FD   │
                  └──────────────────┘

                        │

                    1 thread


Suppose:

    Client A → sending data

    Client B → sending nothing

    Client C → sending data
```

```
poll() returns:

    fd A → ready

    fd B → not ready

    fd C → ready

Thread handling:

    A → processed

    C → processed
```

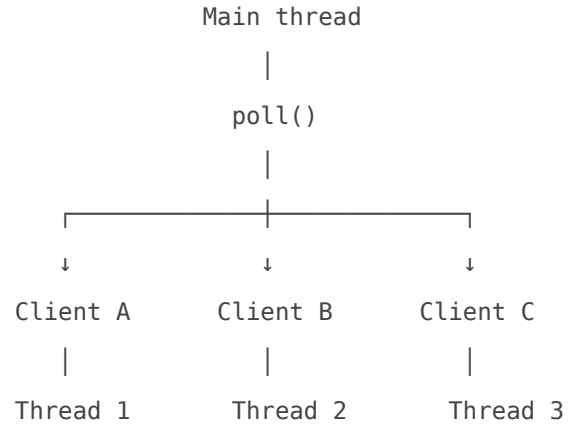
Advantages:

- No synchronization issues between worker threads
- Resource saving: 1 thread carries thousands of clients instead of wasting 1 thread for each client.

Disadvantages:

If during that sequential processing, one FD encounters a time-consuming task (e.g., reading/writing a large file, heavy computation, or blocking somewhere), all other FDs waiting in the queue will have to endure "waiting in line" (getting stuck) until the previous task finishes completing.

13.2 Multi-thread + select/poll/epoll



How it works: Still uses epoll/poll to monitor multiple FDs, but instead of a single thread doing everything, the system has a Main Thread dedicated to accepting connections/events, and then offloads data processing to a Worker Thread Pool (multiple threads running in parallel).

Advantages:

- Resolves the weakness of the single-thread model: If a client requires a heavy task, it is pushed to another thread without blocking the main event-reception thread.
- Maximizes multi-core CPU utilization.

Disadvantages:

- Much more complex: Requires the use of mutexes, queue management, and dealing with the risks of race conditions and deadlocks.

14 Name & Address Conversion: DNS, gethostbyname, getaddrinfo

14.1 DNS

DNS (Domain Name System) is essentially the "global phonebook" of the Internet.

It performs the task of:

```
Translating from a domain name (easy for humans to remember) → an IP address (numeric format for computers to understand and locate each other on the network).
```

14.2 gethostbyname

Nature and Core Limitations (gethostbyname):

- Function: Takes a hostname string (domain name or machine name) and returns detailed information about that machine.
- Critical Limitation: It only supports IPv4 addresses (AF_INET) and is completely helpless with IPv6.
- Standardization Status: The POSIX standard has issued warnings that this function may be deprecated/removed in the future to force developers to switch to the modern, protocol-independent function (getaddrinfo). However, it still widely exists in legacy codebases.

```
struct hostent *gethostbyname(const char *hostname);
```

```
struct hostent {
    char  *h_name;       /* official (canonical) name of host */
    char **h_aliases;    /* pointer to array of pointers to alias names */
    int    h_addrtype;   /* host address type: AF_INET */
    int    h_length;     /* length of address: 4 */
    char **h_addr_list;  /* ptr to array of ptrs with IPv4 addrs */
};
```

Example

```
int main(int argc, char **argv) {
    // If the user does not pass an argument, default to looking up "google.com"
    const char *hostname = (argc > 1) ? argv[1] : "google.com";

    printf("Resolving domain name: %s\n", hostname);

    // Call the gethostbyname function
    struct hostent *he = gethostbyname(hostname);
    if (he == NULL) {
        // h_errno is the variable storing the specific error code for gethostbyname
        herror("gethostbyname error");
        return 1;
    }

    // Print basic information
    printf("Canonical name: %s\n", he->h_name);
    printf("Address type: %d (AF_INET = %d)\n", he->h_addrtype, AF_INET);
    printf("Address length: %d bytes\n", he->h_length);

    // Iterate through the list of IP addresses (since a domain name can point to multiple
IPs)
    struct in_addr **addr_list = (struct in_addr **)he->h_addr_list;
    for (int i = 0; addr_list[i] != NULL; i++) {
        // Convert from binary format to dotted-decimal string format (e.g., "142.250.190.46")
        char ip_str[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, addr_list[i], ip_str, sizeof(ip_str));
        printf(" -> IP address [%d]: %s\n", i + 1, ip_str);
    }

    return 0;
```

```
}
```

Output:

```
./gethostbyname

Resolving domain name: google.com
Canonical name: google.com
Address type: 2 (AF_INET = 2)
Address length: 4 bytes
 -> IP address [1]: 142.250.198.46

Resolving domain name: youtube.com
Canonical name: youtube.com
Address type: 2 (AF_INET = 2)
Address length: 4 bytes
 -> IP address [1]: 142.250.198.206

Resolving domain name: github.com
Canonical name: github.com
Address type: 2 (AF_INET = 2)
Address length: 4 bytes
 -> IP address [1]: 20.205.243.166
```

14.3 getaddrinfo

Protocol-independent: Completely resolves the fatal weakness of gethostbyname (which only supports IPv4). getaddrinfo handles both IPv4 and IPv6 smoothly.

Versatile (2-in-1): Simultaneously performs domain name <-> IP address conversion and service name conversion (e.g., "http", "domain") <-> port number.

Returns a ready-to-use socket structure: Instead of returning a raw list of addresses, it returns a complete sockaddr structure that can be plugged straight into functions like socket(), connect(), or bind() without complex parsing logic.

```
int getaddrinfo(const char *hostname, const char *service,
const struct addrinfo *hints, struct addrinfo **result);
```

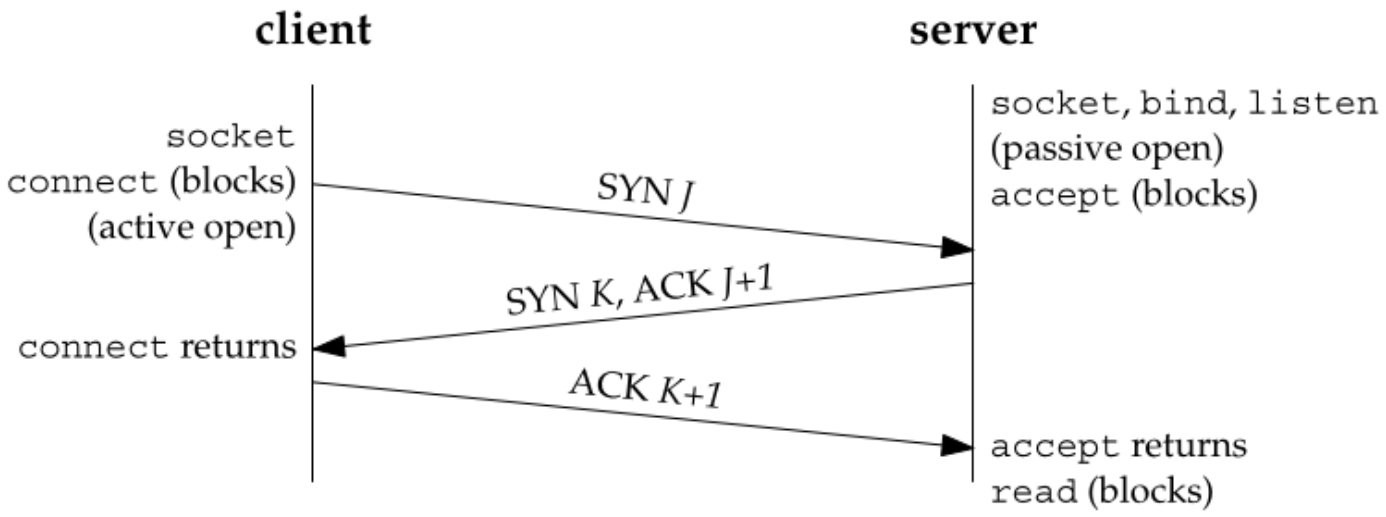
```
struct addrinfo {
    int              ai_flags;      /* AI_PASSIVE, AI_CANONNAME */
```

```
    int              ai_family;     /* AF_xxx */
    int              ai_socktype;   /* SOCK_xxx */
    int              ai_protocol;   /* 0 or IPPROTO_xxx for IPv4 and IPv6 */
    socklen_t        ai_addrlen;    /* length of ai_addr */
    char            *ai_canonname;  /* ptr to canonical name for host */
    struct sockaddr *ai_addr;       /* ptr to socket address structure */
    struct addrinfo *ai_next;       /* ptr to next structure in linked list */
};
```

15 Transport Layer theory: TCP three-way handshake and termination, TIME_WAIT state, Port Numbers

15.1 TCP three-way handshake

1. Three-way handdshake



Server side: Executes socket, bind, and listen (passive open) and blocks at the accept function waiting for a connection.

Client side: Initializes a socket and calls the connect function (active open), sending a SYN J packet to the server.

Handshake process:

Step 1: The client sends the SYN J packet.

Step 2: The server receives it, responding with a SYN K, ACK J+1 packet. At this point, the connect function on the client side is unblocked and returns (connect returns).

Step 3: The client sends an acknowledgement ACK K+1 packet back to the server. At this point, the accept function on the server side returns (accept returns) and transitions to blocking at the read data function (read blocks).

---

2. Termination (4-way handshake)

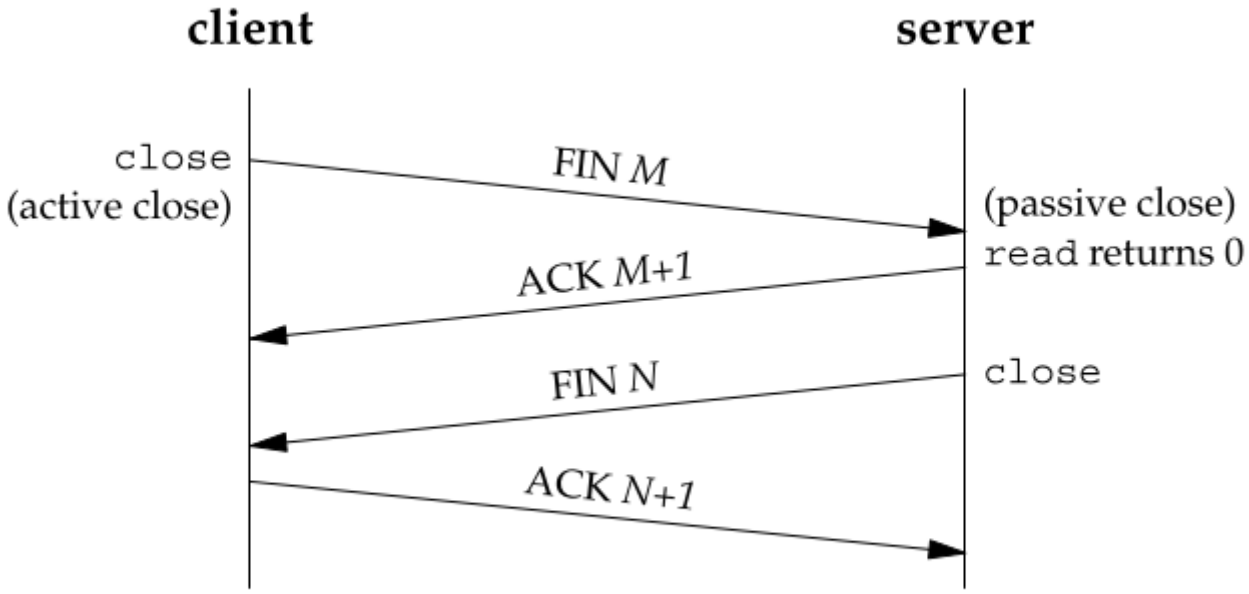


Figure 2.3 Packets exchanged when a TCP connection is closed.

Step 1: The client calls close (active close) and sends a FIN M packet to the server.

Step 2: Upon receiving it, the server's read function returns 0 (signaling that the client has disconnected) and the server sends an ACK M+1 packet back to the client to acknowledge.

Step 3: After finishing its internal processing, the server proceeds to call close (passive close) and sends a FIN N packet toward the client.

Step 4: The client receives it and sends the final acknowledgement ACK N+1 packet back to the server to complete the connection closure.

Note: reason whyin step 3 Server not send FIN N with ACK M + 1 to client

The reason these two packets are not combined into one is because they are triggered by two different entities:

- ACK Packet (Sent IMMEDIATELY by the Kernel): Upon receiving the FIN from the client, the server's kernel automatically responds with an ACK right away to signal "I have received the request," preventing the client from continuously retransmitting due to a suspected network drop.
- FIN Packet (Sent SUBSEQUENTLY by the Application): Even though the kernel has acknowledged with an ACK, the server program might still be busy processing pending data. Only when the server process executes the close() statement does the kernel officially close the socket and send out the FIN packet to close the reverse direction.

15.2 TIME_WAIT state

Responsible Party: The entity performing the active close (active close side) must go through this state.

MSL: The maximum time an IP packet can wander around in the network before it gets dropped.

```
TIME_WAIT = 2 x MSL, lasting from 1 to 4 minutes.
```

Advantage of TIME_WAIT

Ensuring reliable connection closure

- If the final ACK packet sent by the active close side gets lost on the way, the other side will send the FIN packet again.
- By keeping the TIME_WAIT state, the active close side still has the info to send that ACK packet one more time. Without this state, it would reply with an RST (Reset) packet, making the server mistake it for an error.

Allowing "stray" packets to be completely destroyed

- Preventing old packets from a previous connection from arriving late and entering a newly established connection (incarnation) with the same IP address and port, which would cause data mixing.
- The $2MSL$ period ensures that all old packets (and their response packets) have completely vanished from the network before a new connection with the same socket configuration is allowed to be set up.

15.3 Port Numbers

Well-known ports (From 1 to 1023):

- Reserved for standard international services (e.g., port 80 for HTTP, 443 for HTTPS, 22 for SSH).
- On Ubuntu Linux, apps that want to open a port in this range must have root (admin) privileges.

Registered Ports (from 1024 to 32767):

- Reserved for common server apps: Third-party software and services installed on Ubuntu usually choose default ports in this range to run their servers (e.g., MySQL database runs on port 3306, PostgreSQL on 5432, Redis on 6379, etc.).

Ephemeral ports (From 32768 to 60999)

- On modern Linux operating systems (including your Ubuntu), the temporary port range that the kernel uses to automatically assign to a client (when calling connect() without binding a port first)

LAB/STUDIO

LAB 1: Write a concurrent TCP echo server (fork-per-connection) plus a client

8.3 Code Exmamle Server - client

LAB 2: Test failure scenarios: server crash, client crash, and correct handling of SIGPIPE

LAB 3: Write a UDP echo server/client

9.2.3 Code example UDP Echo Server/Client

LAB 4: Build a chat-room server first using select(), then rebuild it using poll(), and compare the code

11.5 Build a chat-room server using select() / poll() and compare

LAB 5: Rebuild the chat-room server once more using epoll (edge-triggered, non-blocking sockets)

12.3 Build the chat-room server once more using epoll (edge-triggered, non-blocking sockets)

LAB 6: Combine multiplexing with a small thread pool (threads + poll/epoll) to handle many simultaneous connections

---

Revision #7
Created 2026-09-28 10:08:35 UTC by Dat Nguyen
Updated 2026-09-30 02:26:19 UTC by Dat Nguyen